Jet Ranger Cockpit User Guide

Setting up, running, wiring, flashing and troubleshooting the Jet Ranger panel

Regenerated 18 October 2026

Contents

1 Overview....4
2 Windows and simulator....5
2.1 Starting up....5
2.2 Running the MSFS bridge....6
2.2.1 Aircraft with estimated dials....7
2.3 Running the FSUIPC bridge (Prepar3D and FSX)....8
2.4 Installing the DodoSim event file (FSUIPC)....8
2.4.1 Event naming....9
2.4.2 Event groups at a glance....9
3 Colour codes and cabling....10
3.1 Reading the diagrams....10
3.2 Colours in the programs....10
3.3 Cabling colour codes....11
3.3.1 Round power connectors....11
3.3.2 PCB connectors....11
3.4 OLED connectors....11
4 Lowrance AirMap GPS....12
4.1 Setting it up....12
4.2 What the GPS is told....12
4.3 Testing without the simulator....13
5 Network....14
5.1 Packet format....15
5.2 What a board does....16
5.3 When the panel loses power....17
5.4 Backlight....18
6 Arduino servo controller....19
6.1 Uploading the firmware....19
6.2 Servos....19
6.3 Other pins....20
6.4 What happens at power-up....20
6.5 Normal running....20
7 Stepper controller (13 Stepper board)....22
7.1 Behaviour....22
7.2 Testing a gauge on demand....23
7.3 Known issues....23
8 OLED dual stepper controller....25
8.1 OLED displays....26
8.2 Behaviour....26
8.3 Known issues....26
9 Radio controller (COM and NAV)....27
9.1 Boards....27

9.2 Inputs and outputs....27
9.3 Data from the PC (UDP 13136)....28
9.4 Commands to the PC (UDP 27001)....28
9.5 Display behaviour....29
9.6 Known issues....29
10 Joystick (HID) interface....30
10.1 How a switch becomes a joystick button....30
10.2 Switch mapping in use....31
10.3 Setting FSUIPC events from the joystick buttons....31
10.4 Known issues....32
11 COM ports and flashing....34
11.1 COM port to Arduino mapping....34
11.2 Flashing a board....35
12 PCBs and wiring....36
12.1 Backlight MOSFET (revisions of 18 October 2026)....37
12.2 Connections on the servo carrier board....37
12.3 Wiring checklist....38
13 Calibration....39
13.1 Calibrating a servo....39
13.2 Calibrating a stepper gauge....39
14 Troubleshooting....40
14.1 Known issues in the code....41
14.2 Open questions....41
15 Reference: UDP tokens....42
15.1 Gauge tokens....42
15.2 Lamp tokens (servo board)....42
15.3 Aircraft information to the GPS interface (UDP 27002)....42
15.4 Status and control tokens....43

1 Overview

The Jet Ranger cockpit drives real gauge needles, warning lights, radio displays and switches from the simulator. A Windows PC exchanges UDP packets over Ethernet with Arduino Mega 2560 panel boards, and a small USB board presents the panel switches to Windows as joysticks.

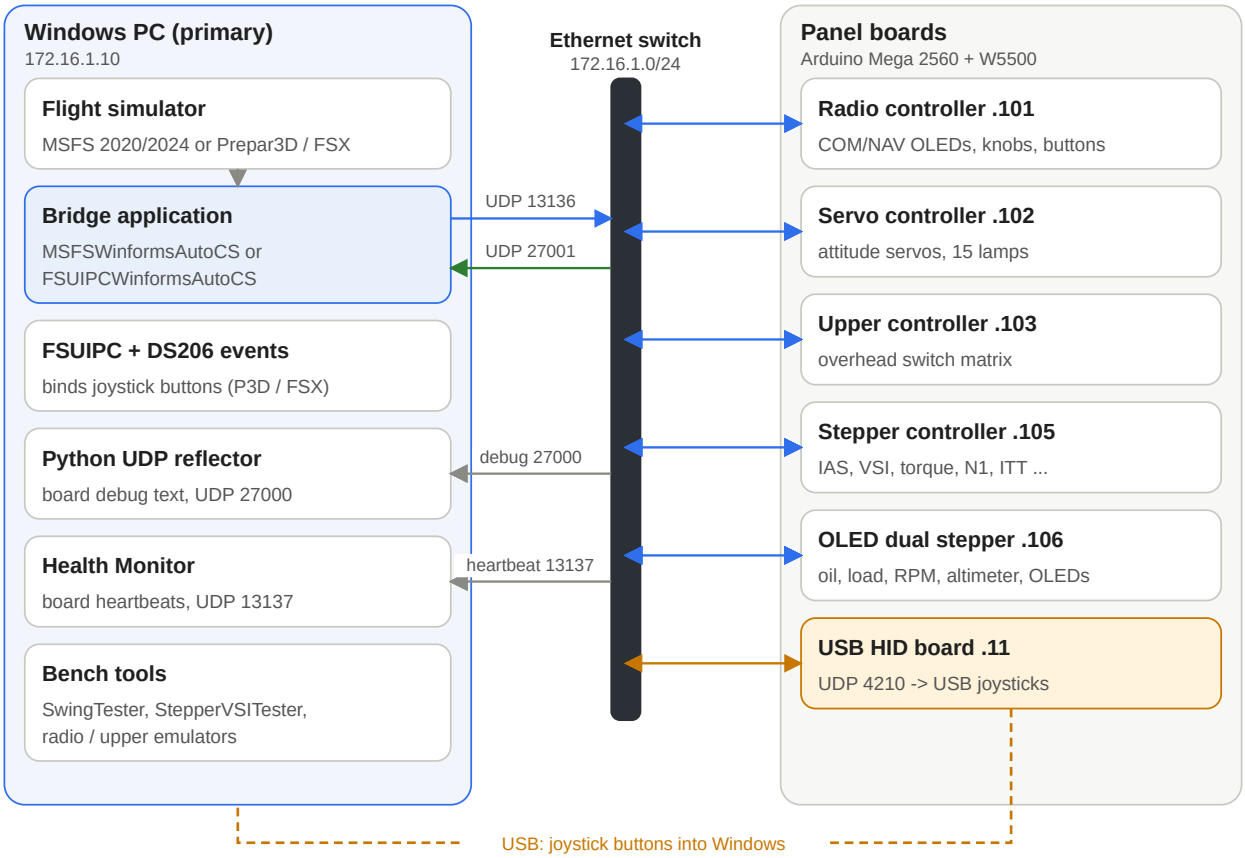


Figure 1 — The whole system

Data flows from the PC to the boards as short text packets. On the primary PC a Python listener shows the boards' debug messages and a separate Windows app, the Health Monitor, tracks their keep-alives. Cockpit switches reach the simulator separately, as joystick buttons (and, in Prepar3D and FSX, FSUIPC events).

Part	Hardware and software	Address	Section
Primary PC	Windows, MSFS 2020 or 2024, or Prepar3D / FSX with DodoSim 206 V2, one bridge program, debug listener, Health Monitor	172.16.1.10	Windows and simulator
Radio controller	Mega 2560 with the Radio Auto Pilot PCB and the COM and NAV panel	172.16.1.101	Radio controller
Servo board	Mega 2560 and Ethernet module on the Front Panel Servo PCB	172.16.1.102	Servo controller
Upper controller	Mega 2560 scanning the overhead switch matrix	172.16.1.103	Joystick (HID) interface
OLED controller	Mega 2560 driving altitude and barometer OLEDs	172.16.1.104	Reference
Stepper board	Mega 2560 on the Jet Ranger 13 Stepper PCB	172.16.1.105	Stepper controller
Dual stepper board	Mega 2560 on the OLED Dual Stepper PCB	172.16.1.106	Dual stepper controller
USB HID board	Leonardo-class board with native USB	172.16.1.11	Joystick (HID) interface

2 Windows and simulator

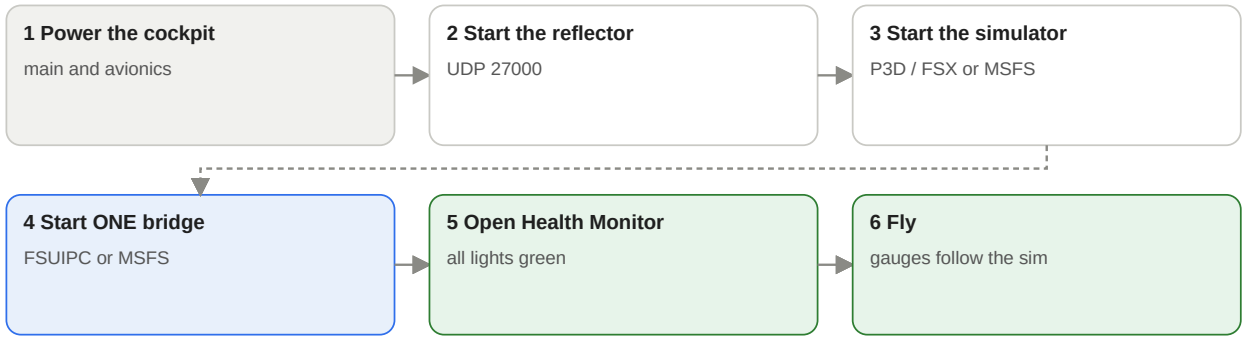
The Windows PC runs the simulator and a **bridge program** that turns simulator variables into UDP text packets for the Arduino boards. Switch inputs travel the other way as joystick buttons.

The bridge sends the Servo, Stepper and Dual Stepper boards each their own packet in **real units** (degrees, knots, feet per minute, psi, percent, degrees C), and the boards convert them with their own calibration. It sends about 10 packets per second to each board; the radio gets a packet whenever a frequency changes and at least every 5 seconds. The Dual Stepper's temperature OLED shows the outside air temperature from the simulator.

Component	Role	Notes
Simulator	Flies the DodoSim 206 or another Bell 206 Jet Ranger	MSFS 2020 or 2024, Prepar3D or FSX
FSUIPCWinformsAutoCS	Bridge for Prepar3D and FSX, reads FSUIPC offsets	Needs FSUIPC
MSFSWinformsAutoCS	Bridge for MSFS, reads SimConnect	Has estimates for the FlyInside B206 and CowaSim Jet Ranger
FSUIPC and DODSIM206V2.EVT	Exposes 234 DodoSim custom events so switches can be bound	Prepar3D and FSX only
Python debug listener	Shows debug messages from the boards	Listens on UDP 27000
Health Monitor	Tracks each board's keep-alive and flags a board that goes quiet	Listens on UDP 13137
LowranceAirMapGps	Feeds a Lowrance AirMap 2000C GPS from the simulator over a serial port	Listens on UDP 27002; see Lowrance AirMap GPS

Run **only one bridge at a time**: both listen for the radio board's commands on UDP 27001. Both bridges also send the aircraft position, altitude, airspeed, heading and simulator date and time to the GPS interface on UDP 27002 (on the same PC).

2.1 Starting up



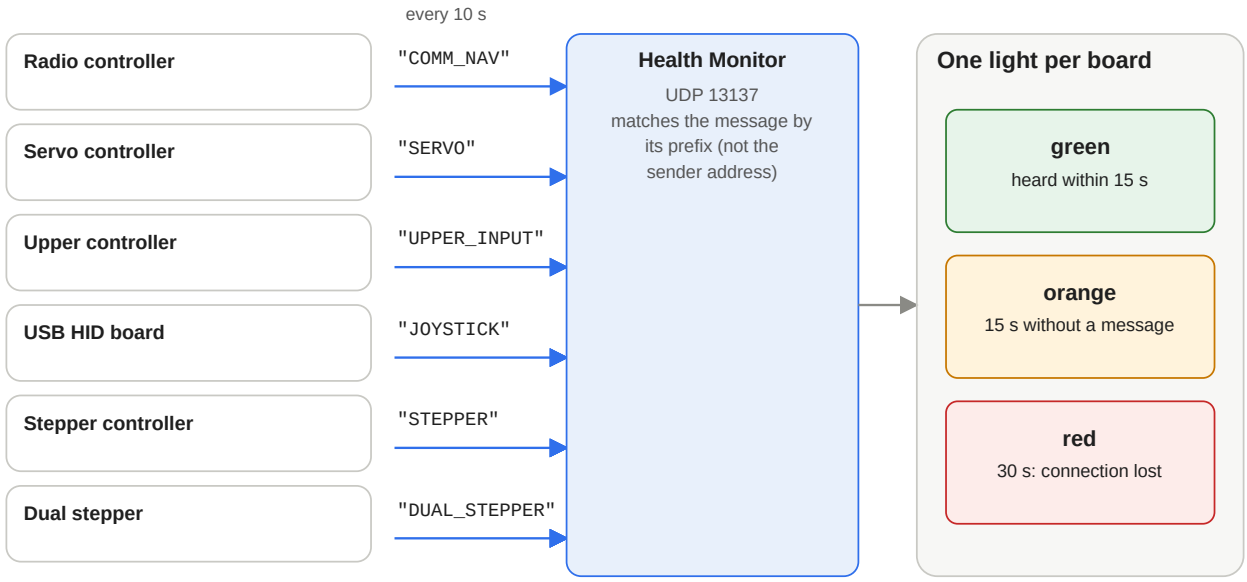
Order matters little: every board waits for data, and the bridge reconnects to the simulator on its own every second.

If a gauge stays at zero for 30 s without data, the board's watchdog parked it on purpose.

Figure 2 — Start-up order

1. Power the cockpit (main and avionics).
2. Start the debug listener (UDP 27000).
3. Start the simulator.
4. Start **one** bridge.
5. Open the Health Monitor: every light should turn green within about 10 seconds.
6. Fly. The gauges follow the simulator.

The order matters little: every board waits for data and the bridge reconnects to the simulator on its own every second. A gauge that stays at zero for 30 seconds without data was parked on purpose by the board's watchdog.

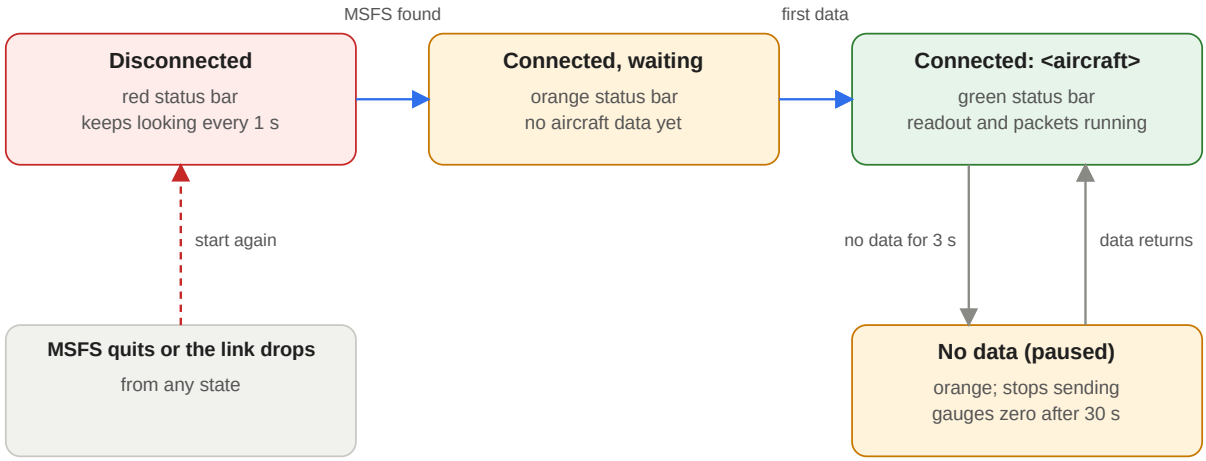


The monitor only listens; it never sends anything. It does not care which bridge is running.

Figure 3 — Health Monitor

2.2 Running the MSFS bridge

1. Build the project `MSFSWinformsAutoCS` (in the repository's `C# Code` folder) and run `MSFSWinformsAutoCS.exe` from its `bin\Debug` folder. Build output is not stored in git, so a fresh copy of the repository has to be built first.
2. Close the FSUIPC bridge if it is running, then start MSFS and load a Jet Ranger. The status bar changes from *Disconnected* (red) to *Connected to MSFS* with the aircraft name (green) once data arrives.
3. The window lists the values being sent and the warning lamps. The Avionics Master box follows the simulator and can also be used to switch it.
4. If the status turns orange and says no data, the simulator is paused or loading and the bridge has stopped sending; the boards return their gauges to zero after 30 seconds and carry on when data returns.
5. Five lamps are driven: engine out, rotor low, low fuel, generator fail (engine speed below 10 percent) and transmission oil pressure (below 40 psi, using the estimated pressure on the FlyInside and CowaSim). The other lamps need aircraft-specific variables. Some MSFS Jet Rangers do not report every value (for example transmission pressure and temperature), so those gauges may stay at zero. With the FlyInside B206 the estimated transmission pressure (35 psi) is below the 40 psi threshold, so that lamp stays on.
6. The status bar shows the aircraft name the simulator reports and follows it when you load a different one.
7. The electrical load follows the size of the total load whichever way the current flows, so with the engine stopped it still shows a load.



A new aircraft is picked up while connected: the name in the status bar changes and the estimate models restart.

SimConnect errors are caught on every path, so a closed simulator never stops the bridge. FSUIPCWinformsAutoCS behaves the same way with FSUIPC.

Figure 4 — Bridge status

2.2.1 Aircraft with estimated dials

Value	Standard SimVars	FlyInside B206	CowaSim Jet Ranger
Rotor, engine speed	**ROTOR RPM PCT,** GENERAL ENG PCT MAX RPM	**L:Aircraft.Rotor.RPM.Percent,** L:...Turbine.N2.Smooth	**standard**
Gas producer (N1)	**TURB ENG CORRECTED N1**	**0.92 x turbine N2**	**standard**
Torque (TQ)	**ENG TORQUE PERCENT**	**collective lever curve,** up to 120 %	**0.42 x SimVar,** smoothed 2.5 s
Turbine out temp (ITT)	**TURB ENG ITT**	**6 x N2 + 1.05 x** (torque - 21)	**EGT - 200 C**
Fuel load (psi)	**ENG FUEL PRESSURE**	**10 x SimVar**	**standard**
Electrical load (%)	**|total amps| / 300 A**	**|total amps| / 300 A**	**|total amps| / 139 A**
Transmission pressure	**SimVar (usually 0)**	**35 psi x rotor fraction**	**48 psi x rotor fraction**
Transmission temp	**SimVar (usually 0)**	**slow warm-up from air temp**	**0.92 x engine oil temp**
Recognised by	**otherwise**	**turbine L:var, or title** contains FlyInside	**title contains 206B3**

Green = the standard SimVar is used unchanged. Estimates come from a few dial readings and may be a few percent off.

Figure 5 — Which values come from where

FlyInside B206. Its own gauges show values that are not in the standard simulator variables: rotor speed, engine speed, gas producer, torque, turbine out temperature, fuel pressure and transmission oil. The bridge reads the aircraft's own rotor and engine speed variables and estimates the rest from cockpit dial readings: torque follows the collective lever, turbine out temperature rises with torque, and the transmission oil pressure and temperature are modelled (the temperature warms up slowly from the outside air temperature and starts again from cold each time the bridge restarts). In this aircraft holding full collective for more than about 2 seconds over-torques the engine and breaks it. Its cockpit load dial stays at about 50 percent whatever the load.



Both curves were fitted to dial readings from screenshots; check them against the cockpit at other power settings.

FlyInside above lever 73.6 climbs at about 1.74 points per percent; the engine over-torques if full collective is held for 2 s.

Figure 6 — Torque estimates

CowaSim Jet Ranger (shown as `206B3` with its registration). Most gauges match the standard variables. Its torque, turbine out temperature and electrical load dials do not, and it has no transmission oil values: the bridge smooths the torque, takes the turbine out temperature from the exhaust gas temperature, rescales the load and estimates the transmission oil from rotor speed and engine oil temperature.

These estimates were fitted to a few dial readings and may be a few percent off at other power settings, so compare the panel gauges with the cockpit dials. Other aircraft use the standard variables.

2.3 Running the FSUIPC bridge (Prepar3D and FSX)

Start FSUIPC with the simulator, then run `FSUIPCWinformsAutoCS.exe`. The status bar shows when it is connected to Prepar3D. The window lists the values being sent and the 21 annunciator bits. The bridge sends about 10 packets per second to each board and listens on UDP 27001 for the radio board's commands (frequency swaps, standby tuning, battery, alternator, avionics master).

2.4 Installing the DodoSim event file (FSUIPC)

Copy `DODSIM206V2.EVT` into the `{P3D or FSX}/Modules` folder, next to `FSUIPC.ini`. Add this to `FSUIPC.ini` if FSUIPC has not added it itself:

```
[EventFiles]
0=DodoSim206
```

Restart the simulator. The `DS206.*` events now appear in FSUIPC's button and key assignment lists. Bind each physical switch to its event. After an EVT update FSUIPC re-indexes events alphabetically, so check and re-map bindings.

2.4.1 Event naming

Suffix	Parameter	Behaviour	Example
`_SET`	Integer (usually 1 = on, 0 = off)	Sets a state directly; best for latching toggle switches	`DS206.BATTERY_SWITCH_SET`
`_TOGGLE`	None	Flips the state each press	`DS206.FUEL_VALVE_SWITCH_TOGGLE`
`_TRIGGER`	None (some take 1 to 10 as a multiplier)	Momentary; tick Control to repeat while held	`DS206.STARTER_BUTTON_TRIGGER`
none	None	One-shot action	`DS206.CLOCK_MODE_BUTTON`

For physical latching switches prefer the `_SET` events: send 1 on switch-on and 0 on switch-off so the simulator never drifts out of step with the panel.

2.4.2 Event groups at a glance

Group	Event numbers	Examples
Circuit breakers	36 to 95, 107 to 110	Hoist, SAS, fuel pumps, governor, ignition, starter
Lights and master switches	96 to 101, 111 to 122, 209 to 212, 224 to 227	Battery, generator, avionics, pitot heat, landing lights
Engine and flight controls	1 to 2, 207 to 208, 213 to 223, 228 to 232	Rotor brake, hydraulics, force trim, starter, governor beep
Instruments	3 to 33	Annunciator test, AI cage, radar altimeter, HSI and OBS knobs
Radios and audio	129 to 192	COM and NAV swap and tuning, ADF, DME, audio panel
Transponder	193 to 206	Mode knob, digit buttons, IDT, VFR
SAS modes	123 to 128	Master, HDG, NAV, SPD, ALT, VRT
Other	102 to 106, 233 to 234	Difficulty rotary, heat and vent, external power

3 Colour codes and cabling

This chapter explains the colours used in the diagrams and programs, and the colour codes used for cables and connectors.

3.1 Reading the diagrams

Each diagram box is shaded to say what kind of item it is. The same shades are used in every diagram.

Colour	What it means in the diagrams
White	An ordinary item: a board, program, step or detail
Blue	The subject of the figure or the main step: the bridge, the Health Monitor, the code that decides
Green	Running or healthy, power on, or an output: connected, the gauge or backlight being driven
Amber	Waiting, a warning or a special path: no data, the default position, the USB HID path
Red	Lost, off, failed or a reset
Gray	Supporting or inactive: notes, items that stay off
Purple	Tells boards or groups apart where the figure says so (for example the Dual Stepper in the Swing ALL sequence, the Upper controller in the PCB map)
Teal	The same, for the ADF controller in the PCB map
Dark	Column headers and the Ethernet switch

Arrows follow the same idea: blue is data going to a board over UDP, green is commands or events coming back, amber is the USB HID path, gray is an ordinary link, and a dashed line is an optional or recovery path.

3.2 Colours in the programs

Where	Colour	Meaning
Health Monitor light	Green	A heartbeat was heard within the last 15 seconds
Health Monitor light	Orange	15 seconds without a heartbeat (connection warning)
Health Monitor light	Red	30 seconds without a heartbeat (connection lost)
Bridge status bar	Red	Disconnected from the simulator
Bridge status bar	Orange	Connected but no data (the simulator is paused or loading)
Bridge status bar	Green	Connected, with the aircraft name
Board status LEDs (pins 14 green, 15 red)	Green and red	Alternate every 200 ms while the sketch runs; red flashes fast while Ethernet starts
SwingTester	Gold	The gauge row being swung, or a warning lamp that is on
GPS bench tester	Pink	A box whose value could not be read (it is not used)
Upper controller emulator	Green	A switch that is on
GPS interface	Green	The position comes from aircraft data over UDP
GPS interface	Orange	No data for 30 seconds: the Brisbane Airport default is being sent

3.3 Cabling colour codes

3.3.1 Round power connectors

Colour	Supply
Red	12 V
Gray	5 V

3.3.2 PCB connectors

Colour	Use	Example
Uncoloured	Digital input	
Blue	Analog	
Red	12 V	Magnetic coils
Green	12 V backlighting	
Yellow	OLED	
Orange	LEDs	

The two 12 V colours are different on purpose: red is 12 V for coils, green is 12 V for backlighting.

3.4 OLED connectors

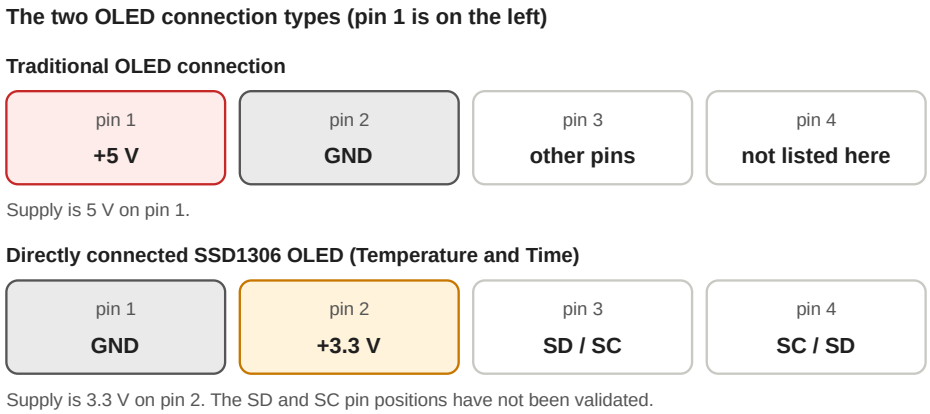


Figure 7 — The two OLED connection types

The OLED connectors are **not the same** on every display:

- **Traditional OLED connection:** +5 V on pin 1 and GND on pin 2.
- **Directly connected SSD1306 OLEDs** (the Temperature and Time displays): GND on pin 1 and +3.3 V on pin 2.

The SD and SC (data and clock) pin positions of the directly connected displays have not been validated. Do not plug one type of cable into the other type of connector: pins 1 and 2 are reversed and the supply voltages differ.

4 Lowrance AirMap GPS

`LowranceAirMapGps` makes the PC the GPS for a **Lowrance AirMap 2000C**. It receives the aircraft position from the bridge and sends it to the GPS over a serial cable as NMEA sentences twice a second.

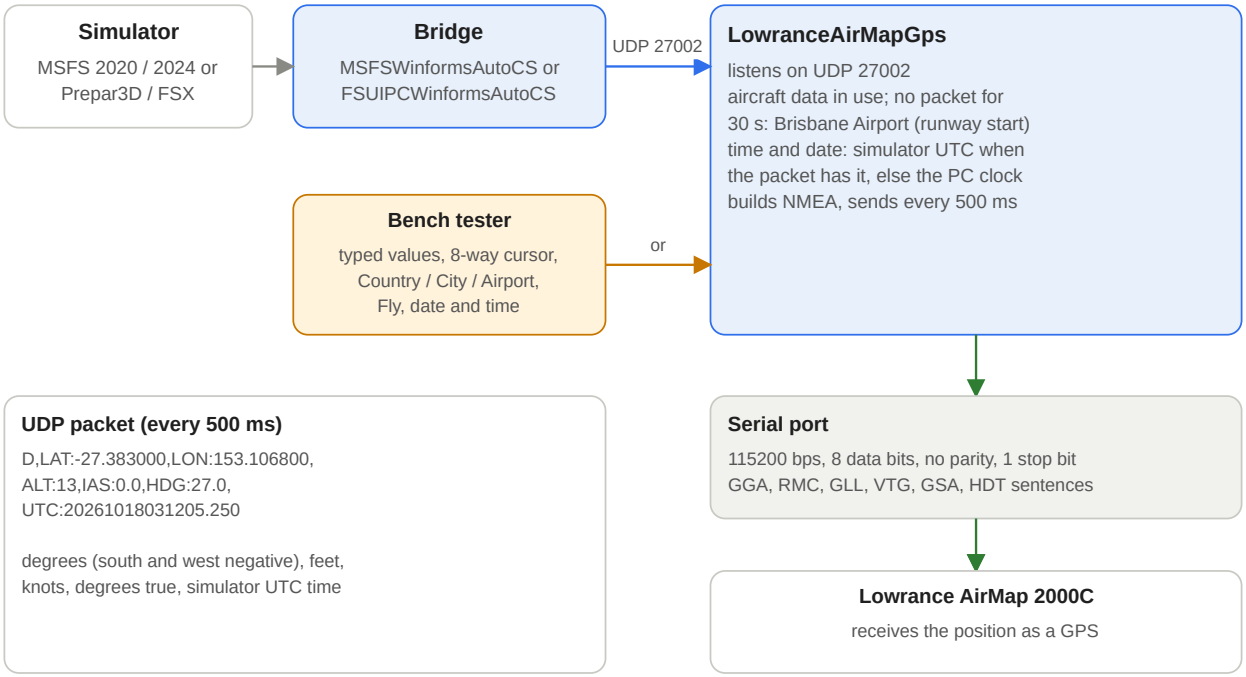


Figure 8 — From the simulator to the GPS

4.1 Setting it up

1. Connect the GPS to the PC with its serial cable (a USB serial adapter works).
2. Run `LowranceAirMapGps.exe`. The first time it asks which serial port the GPS is on; it remembers the answer (*Change serial port...* changes it later). The port runs at **115200 bps, 8 data bits, no parity, 1 stop bit**.
3. Start the simulator and one bridge on the same PC. The bridge sends the aircraft information to UDP port 27002 without any setting; if the GPS interface runs on a different PC, change `GpsHost` in the bridge's `Form1.cs`.
4. The window shows the serial port state, the number of packets received, where the position comes from, the values being sent and the time source. The sentences being sent are listed at the bottom.

4.2 What the GPS is told

- **Position, altitude, airspeed and heading** from the simulator, sent as GGA, RMC, GLL, VTG, GSA and HDT sentences. NMEA has no airspeed sentence a GPS accepts, so the airspeed is sent as the GPS speed.
- **Date and time** are the simulator's UTC, ticking in real time between packets. If the packets have no time, or nothing has arrived for 30 seconds, the PC clock is used. The window says which.
- **No data for 30 seconds** (the simulator is paused, closed or not running): the GPS is told it is at Brisbane Airport, at the start of the runway, speed 0. It returns to the live position as soon as packets arrive again.

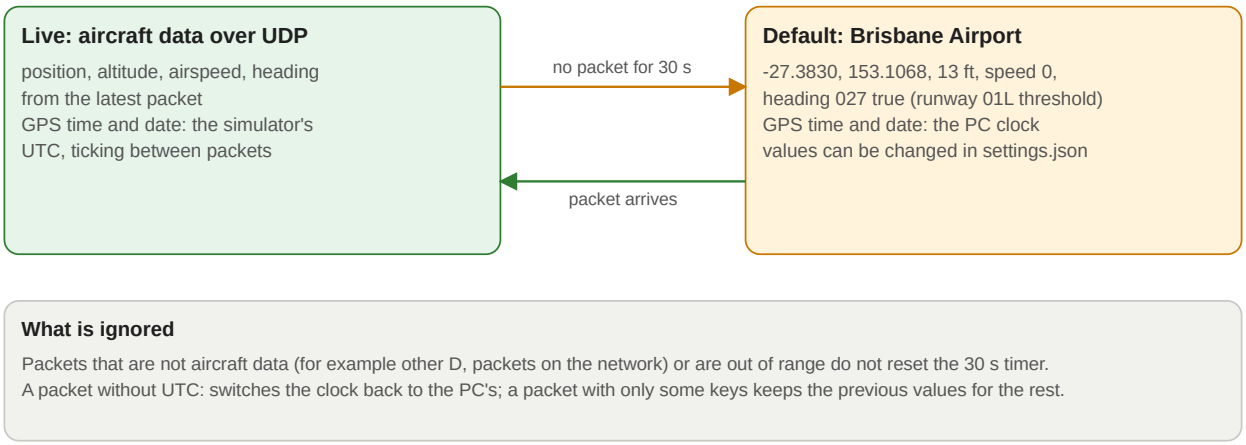


Figure 9 — Live data and the Brisbane default

The default position and the UDP port can be changed in `%AppData%\LowranceAirMapGps\settings.json`. The Brisbane coordinates came from SkyVector and were not checked against the AIP.

4.3 Testing without the simulator

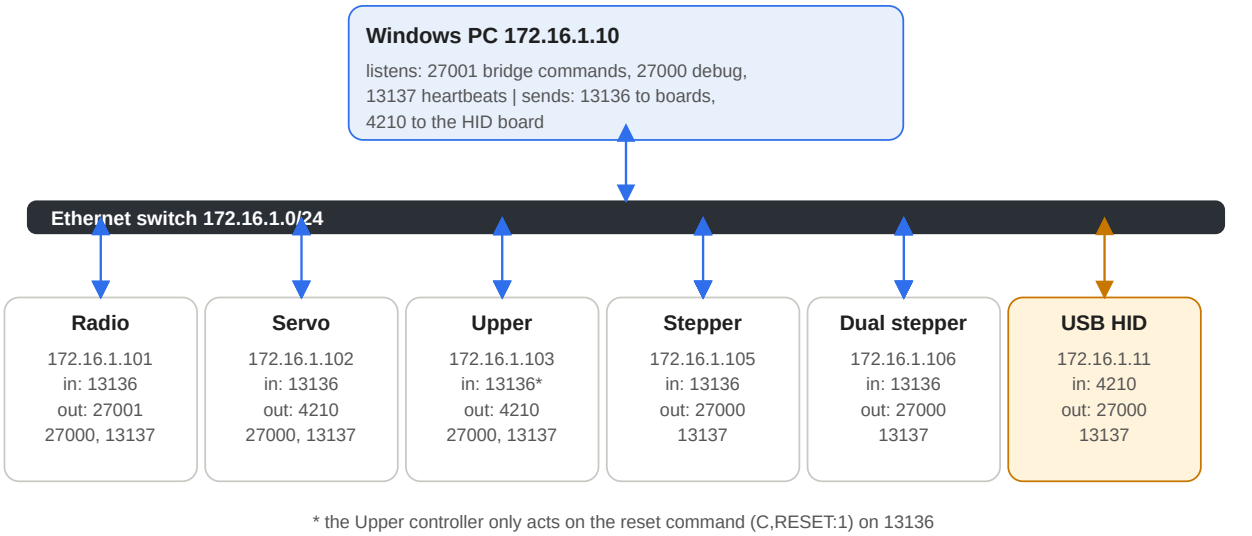
`LowranceAirMapGpsBenchTester` sends the same packets from a window:

- Choose a **Country, City and Airport** to put the aircraft on the ground at that airport.
- Type the **latitude, longitude, altitude, speed and heading**, and press Enter after each.
- Use the **8-way cursor** (the buttons or the arrow keys) to move the position in steps (default 100 m); hold to repeat.
- Tick **Fly** to move along the heading at the current speed; **Time x** speeds it up (1 to 100).
- Set the **date and time (UTC)** the GPS should show, or press *PC clock*.

Use one sender at a time (a bridge or the tester): a second one just overwrites the position. The GPS interface has not yet been tested with the real AirMap 2000C, so check that the GPS shows the position and that it accepts the 115200 bps rate.

5 Network

All devices sit on one wired `172.16.1.x` subnet with fixed addresses; there is no DHCP on the boards.



* the Upper controller only acts on the reset command (C,RESET:1) on 13136

What the ports carry

13136	data (D,...) and control (C,...) packets from the PC to a board
27001	radio, battery and alternator commands from the Radio board to the bridge
4210	switch events (index,state) and key events to the USB HID board; also C,RESET:1 for it
27002	aircraft information from a bridge (or the bench tester) to the Lowrance GPS interface
27000	debug text from every board to the UDP reflector
13137	10 second heartbeat text from every board to the Health Monitor

Figure 10 — Who talks to whom

Device	IP address	MAC	Keep-alive text
Primary PC (simulator, bridge, debug listener, Health Monitor)	172.16.1.10		
USB HID board	172.16.1.11	DE:AD:BE:EF:FE:ED	`JOYSTICK`
Radio controller	172.16.1.101	A8:61:0A:9E:83:01	`COMM_NAV`
Servo board	172.16.1.102	A8:61:0A:9E:83:02	`SERVO`
Upper controller	172.16.1.103	A8:61:0A:9E:83:03	`UPPER_INPUT`
OLED controller	172.16.1.104	A8:61:0A:67:83:68	none
Stepper board	172.16.1.105	A8:61:0A:67:83:69	`STEPPER`
Dual stepper board	172.16.1.106	A8:61:0A:67:83:6A	`DUAL_STEPPER`

Every live board has its own IP and MAC address. The firmware sends its debug and keep-alive traffic to `172.16.1.10` (`reflectorIP`), so the primary PC must hold that address, or `reflectorIP` must be changed and the sketch uploaded again.

UDP port	Direction	Purpose
13136	PC to boards	Gauge, lamp and radio data (`D,`) and control (`C,`) packets
27001	Radio controller to PC bridge	Standby frequencies and commands such as `COM1_RADIO_SWAP`
27002	Bridge to the GPS interface (same PC)	Aircraft position, altitude, airspeed, heading and simulator time, every 500 ms
4210	Servo and Upper controllers to USB HID board	Switch changes (`index,state`) and keypad keys (`KEY,key,state`)
13137	Boards to Health Monitor	Keep-alive text every 10 seconds
27000	Boards to debug listener	Debug messages, each prefixed with the board name

5.1 Packet format

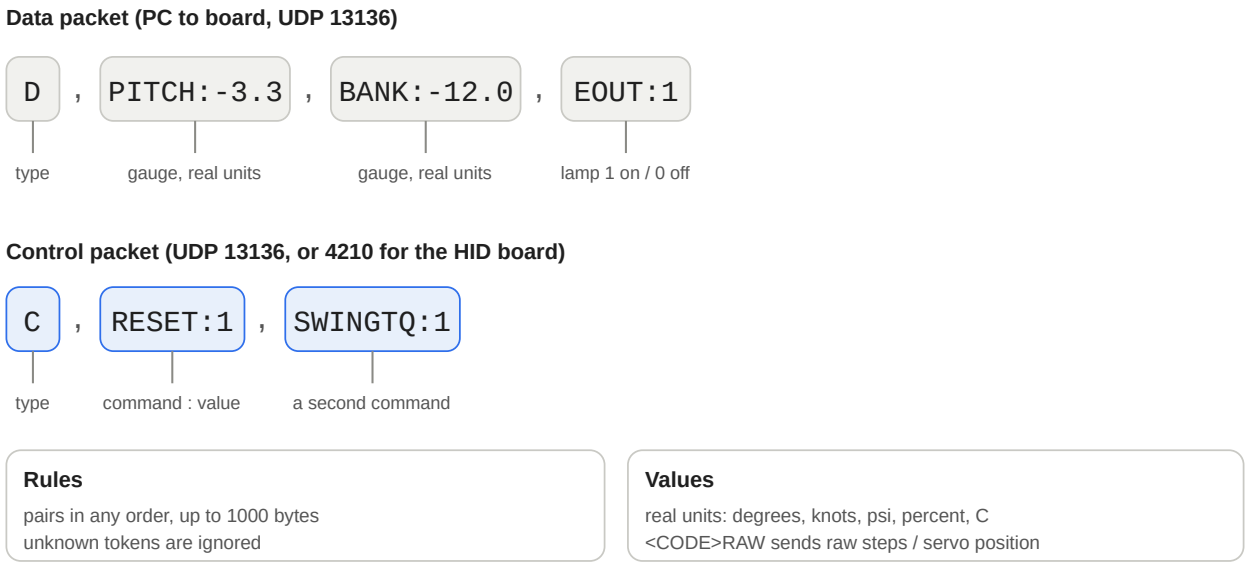


Figure 11 — Data and control packets

A packet can carry any number of pairs, in any order, up to 1000 bytes. Unknown tokens are ignored, so the PC can send only the values that changed. For example `D,PITCH:5,BANK:-10,LOWF:0` sent to the servo board sets 5 degrees pitch, minus 10 degrees bank and turns off Low Fuel.

To test without the simulator, send a packet from a PC on the subnet, for example with PowerShell:

```
$u = New-Object System.Net.Sockets.UdpClient
$b = [Text.Encoding]::ASCII.GetBytes('D,PITCH:5,RLOW:1')
$u.Send($b, $b.Length, '172.16.1.102', 13136)
```

5.2 What a board does

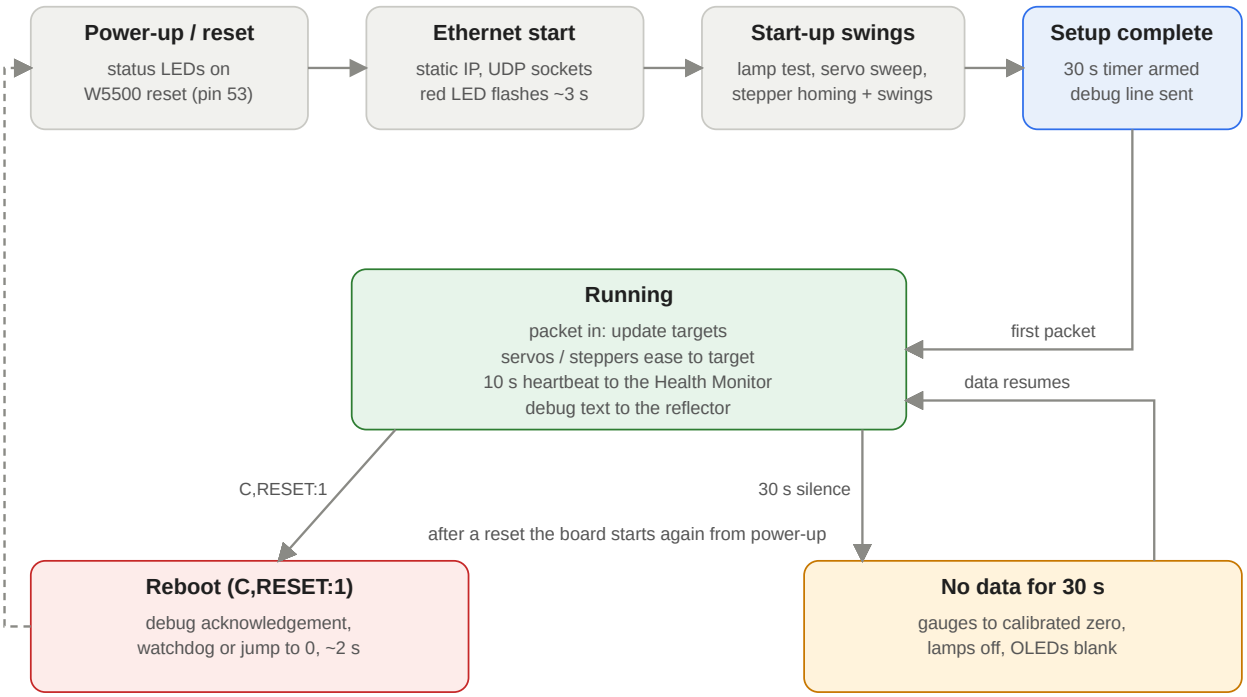


Figure 12 — Board life cycle

After start-up and its self-test a gauge board checks UDP 13136 every 5 ms and moves each needle smoothly towards its target. It sends its keep-alive every 10 seconds. If no packet arrives for **30 seconds** every needle returns to zero (the Servo board also switches its lamps off, the Dual Stepper blanks its OLEDs and the Radio blanks its displays); the next packet brings everything back.

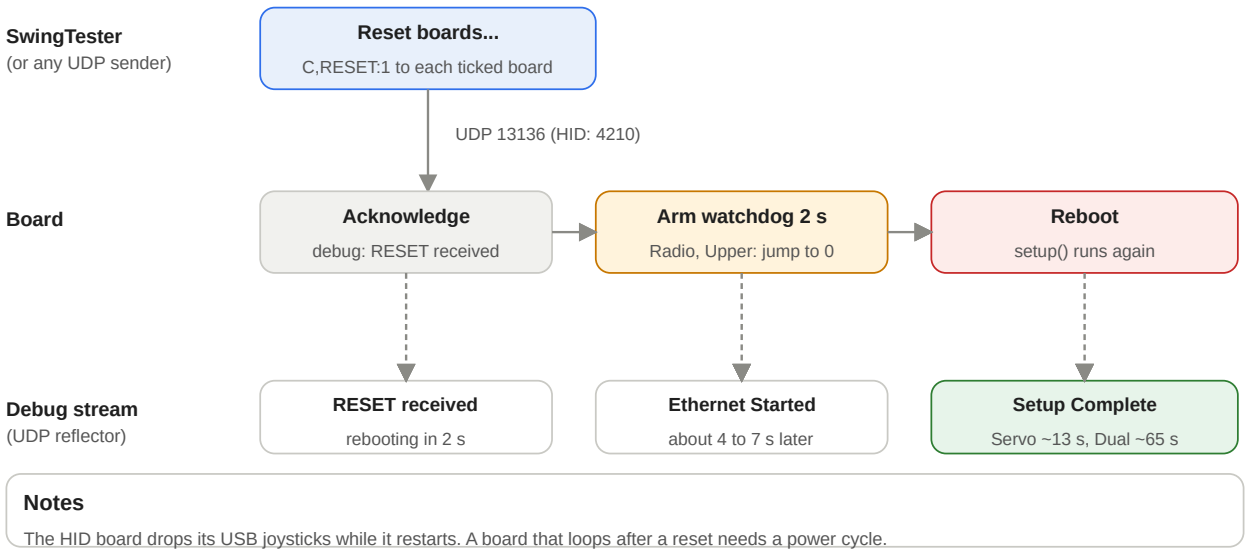


Figure 13 — Remote reset

The control packet `C,RESET:1` reboots a board about 2 seconds later. The SwingTester's *Reset boards...* button sends it to the boards you tick.

5.3 When the panel loses power

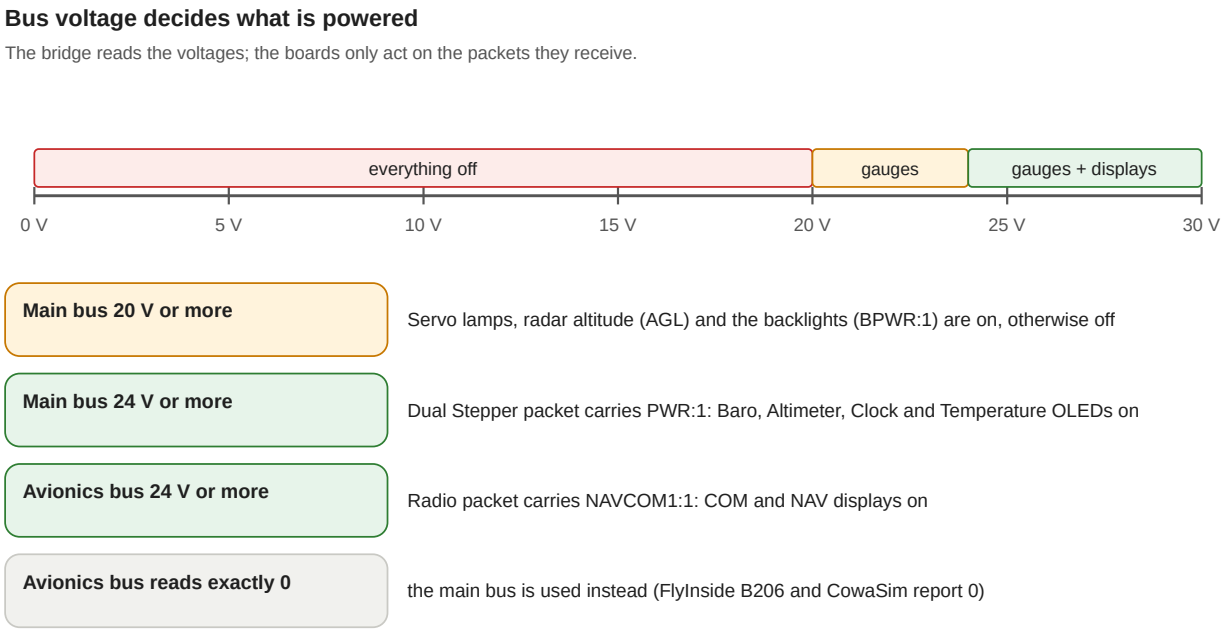


Figure 14 — Bus voltage rules

The bridge turns parts of the panel off when the simulated electrics drop: lamps, radar altitude and the backlights below 20 V on the main bus, the Dual Stepper's four OLEDs below 24 V on the main bus, and the Radio's COM and NAV displays below 24 V on the avionics bus (the main bus is used if the aircraft reports the avionics bus as 0).

5.4 Backlight

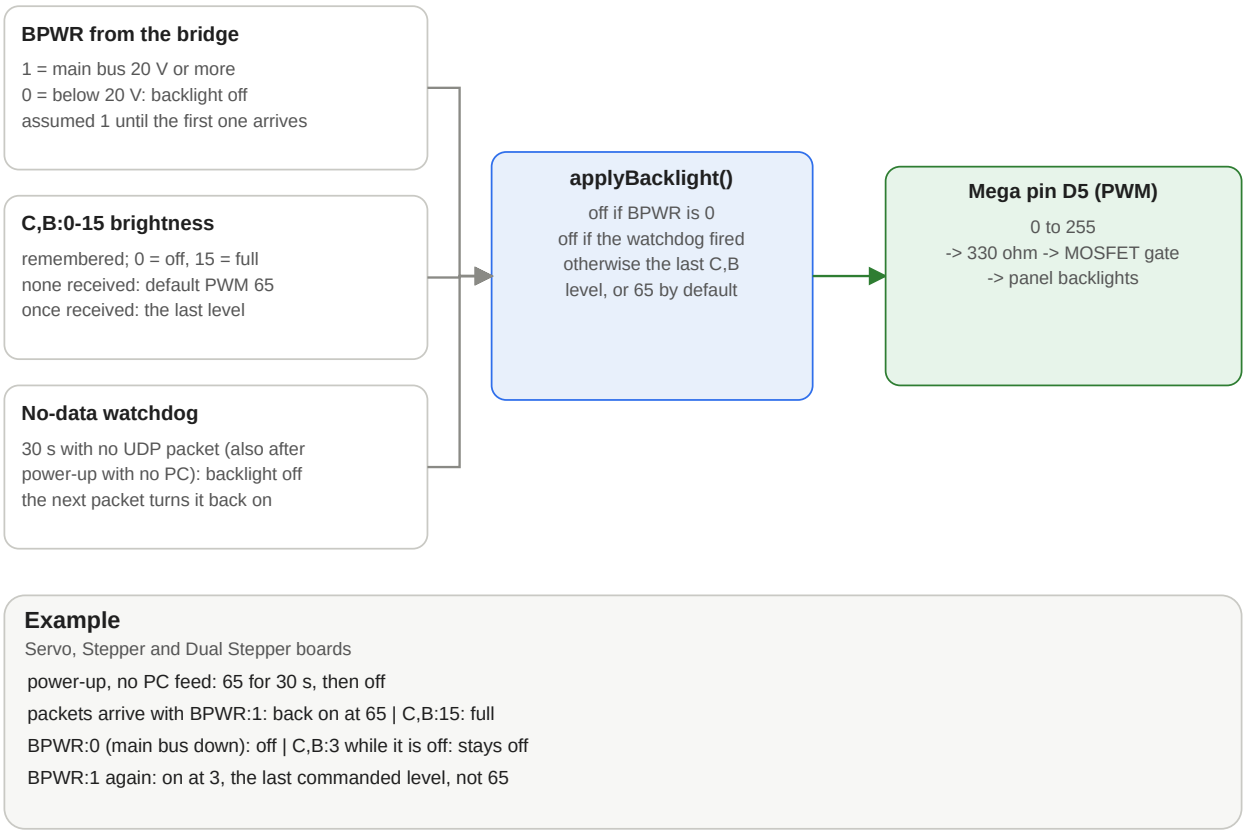


Figure 15 — How the backlight level is decided

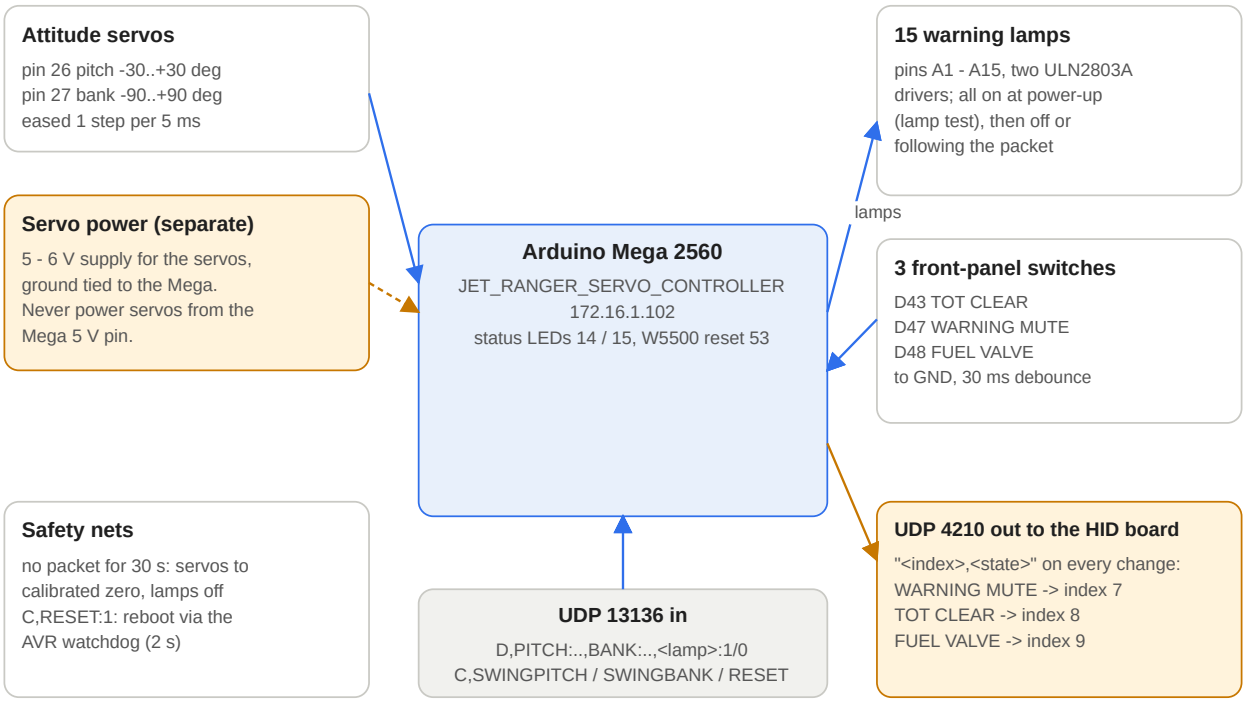
The Servo, Stepper and Dual Stepper boards drive their panel backlights from Mega pin D5 through a MOSFET on the PCB. Three things decide the level:

- **Main bus power (`BPWR`).** The bridge sends `BPWR:1` while the main bus is 20 V or more and `BPWR:0` below that, which switches the backlight off. Until the first `BPWR` arrives the board leaves the backlight on.
- **Brightness (`C,B:0` to `C,B:15`).** 0 is off and 15 is full. The board remembers the last level. If no brightness command has been sent the backlight uses a default level (PWM 65); once one has been sent, every time power returns the backlight comes back at the **last level you set**, not the default. A level sent while the bus is down is applied when power returns.
- **No data.** After 30 seconds with no UDP packet, including after the board is powered up with no PC sending, the backlight goes off along with the gauges. The next packet turns it back on.

The **SwingTester** has a brightness slider that sends the level to every board except the USB HID board, and *Backlight power ON / OFF* buttons for testing the main bus rule. The Radio and Upper controllers accept the command but do not drive a backlight yet.

6 Arduino servo controller

The servo board is an Arduino Mega 2560 with an Ethernet module, on the Jet Ranger Front Panel Servo PCB at 172.16.1.102. It drives the attitude indicator's two servos, switches the 15 caution and warning lamps and reads three front-panel switches. It runs `JET_RANGER_SERVO_CONTROLLER.ino`. The other gauges that were once servo-driven now run on the two stepper boards.



Joystick 1 buttons 8, 9 and 10 in Windows (the firmware counts from 0). FSUIPC needs them bound to DS206 events.

Calibration: PITCH_DEG_TABLE -30 deg 179, 0 deg 113, +30 deg 70; BANK_DEG_TABLE -90 deg 5, 0 deg 93, +90 deg 179 (servo positions).

Figure 16 — Servo board

6.1 Uploading the firmware

1. Open the sketch in the Arduino IDE and select *Board: Arduino Mega or Mega 2560* (or use `arduino-cli`, see Flashing a board).
2. Install the libraries it uses: Ethernet (with SPI), Servo, Ewma (jonnieZG) and DCS-BIOS.
3. Upload, then watch the start-up self-test: both servos sweep and all 15 lamps light.

6.2 Servos

Instrument	Mega pin	UDP token	Range
Attitude bank (roll)	27	`BANK`	-90 to +90 degrees
Attitude pitch	26	`PITCH`	-30 to +30 degrees

Both tokens take attitude in degrees, with one decimal place allowed (`D,PITCH:4.5,BANK:-12`). The board turns degrees into a servo position with the tables in Calibration. `BANKRAW` and `PITCHRAW` take a raw servo position (0 to 180) and skip the tables. The PCB still has 16 servo headers from the earlier design; only pins 26 and 27 are used.

6.3 Other pins

Function	Pin	Behaviour
Green status LED	14	Flashes alternately with red every 200 ms while running
Red status LED	15	Fast flash during the 3 second Ethernet start-up, then alternates with green
Ethernet module reset	53	Pulsed low for 2 ms at power-up
Backlight	5	PWM to the backlight MOSFET (`BACKLIGHT_PWM`); see Backlight
Lamps	A1 to A15	Digital outputs, HIGH = lit (see Reference)
TOT clear switch	43	Switch to GND, debounced 30 ms; each change goes to the USB HID board as Joystick 1 button 9 (index 8)
Warning mute switch	47	The same, Joystick 1 button 8 (index 7)
Fuel valve switch	48	The same, Joystick 1 button 10 (index 9)

The 15 warning lamps on the Servo board

Green: the bridge drives it from the simulator. Grey: needs aircraft-specific variables, so it stays off.

ENGINE OUT pin A1 token EOUT engine speed < 60 %	**ROTOR RPM LOW** pin A2 token RLOW rotor <= 95 %	**XMSN OIL PRESS** pin A3 token TOPW pressure < 40 psi
XMSN OIL TEMP pin A4 token TOTW always off	**BATTERY HOT** pin A5 token BHOT always off	**BATTERY TEMP** pin A6 token BTMP always off
ENGINE CHIP pin A7 token EC always off	**TAIL ROTOR CHIP** pin A8 token TRC always off	**XMSN CHIP** pin A9 token TC always off
BAGGAGE DOOR pin A10 token BD always off	**AFT FUEL FILTER** pin A11 token FFLTR always off	**FUEL PUMP** pin A12 token FPMP always off
GENERATOR FAIL pin A13 token GENF engine speed < 10 %	**LOW FUEL** pin A14 token LOWF fuel <= 17 gal	**SC FAIL** pin A15 token SCF always off

All lamps go out when the main bus is below 20 V. Sent as <token>:1 / <token>:0 in every Servo packet.

The lamp test at power-up lights all 15; a no-data timeout turns them all off.

Figure 17 — The 15 lamps and their pins

6.4 What happens at power-up

1. All 15 lamps switch on as a lamp test.
2. The Ethernet module is reset and starts at 172.16.1.102; the red LED flashes for 3 seconds.
3. *Ethernet Started* goes to the debug listener.
4. Pitch, then bank, sweeps from minimum to maximum and back (about 2 seconds each) and parks at its calibrated level position.
5. All lamps switch off.
6. The starting state of the three switches is sent to the USB HID board, *Setup Complete* is logged and the 30 second watchdog starts.

6.5 Normal running

- Every 5 ms the board checks UDP 13136 and stores each value as a new target; each servo steps one position towards its target, so the attitude indicator moves smoothly.
- A servo that has not moved for 1 second is detached to stop buzz; the next movement re-attaches it.

- Lamps change as soon as a 0 or 1 arrives.
- `C,SWINGPITCH:1` or `C,SWINGBANK:1` re-runs one sweep on demand (the board pauses for about 2 seconds).

7 Stepper controller (13 Stepper board)

A second Arduino Mega 2560, on the Jet Ranger 13 Stepper PCB at 172.16.1.105, drives X27-style stepper gauges directly from its pins. Unlike the servo board it takes real units and converts them to needle steps with calibration tables in `JET_RANGER_STEPPER_CONTROLLER.ino`.

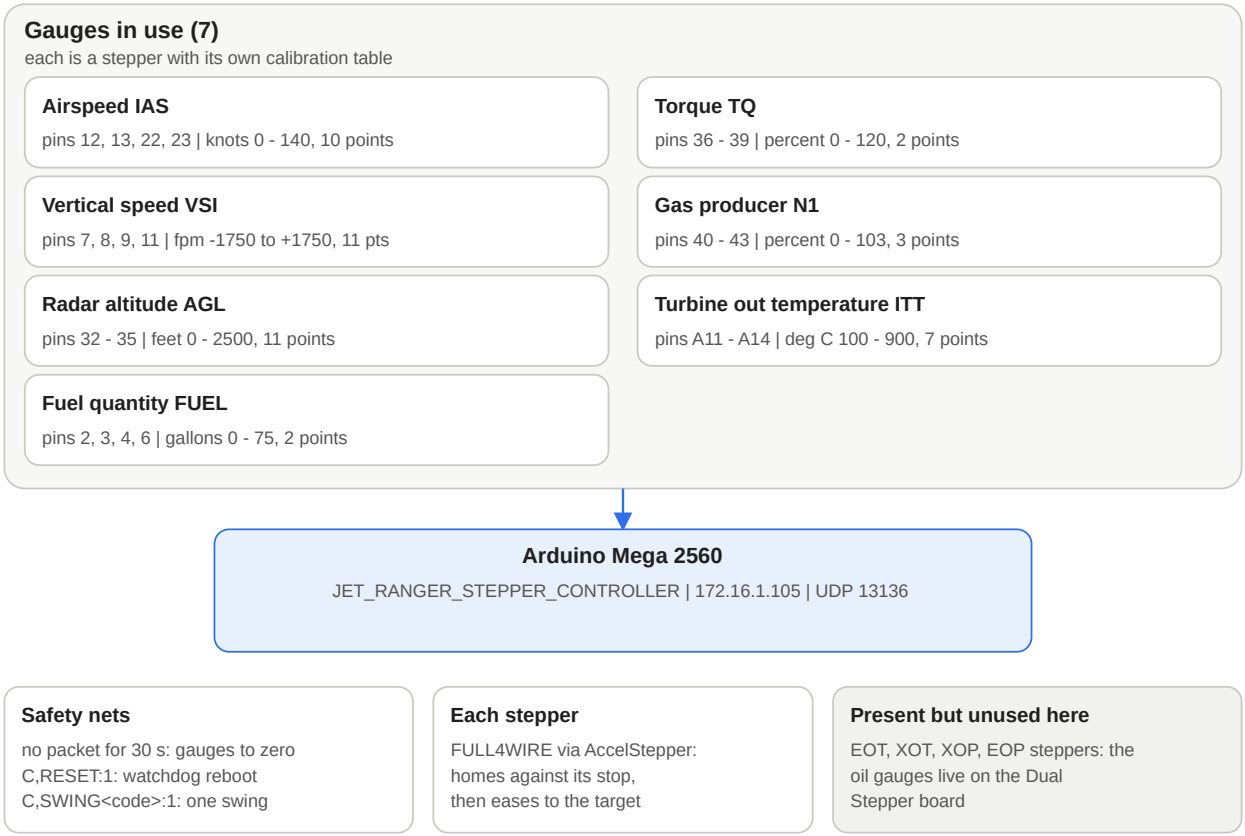


Figure 18 — Stepper board

Gauge	Token	Mega pins	Units	Calibration
Airspeed	`IAS`	12, 13, 22, 23	knots 0 to 140	10-point table
Vertical speed	`VSI`	7, 8, 9, 11	ft/min, plus and minus 1750	11-point table
Radar altitude	`AGL`	32 to 35	feet 0 to 2500	11-point table
Torque	`TQ`	36 to 39	percent 0 to 120	2-point table
Gas producer	`N1`	40 to 43	percent 0 to 103	3-point table
Turbine out temperature	`ITT`	A11 to A14	degrees C 100 to 900	7-point table
Fuel quantity	`FUEL`	2, 3, 4, 6	US gallons 0 to 75	2-point table

Every token has a raw twin with `RAW` on the end (for example `IASRAW:300`). It moves the needle by raw steps and skips the table, which is how new calibration points are measured on the bench. The board also has four oil and transmission steppers with placeholder scales; those gauges are driven from the Dual Stepper, so wire each physical gauge to one board only. Pins 24 to 31 are free.

7.1 Behaviour

- **Power-up:** status LEDs flash, Ethernet starts, backlighting ramps up, then each gauge does a self-test sweep in turn: fuel, torque, turbine out, gas producer, VSI, airspeed, radar altitude. Each sweep has its own on/off switch (`SwingFA`, `SwingTQ` and so on) near the top of the sketch.
- **Running:** every 5 ms the board checks UDP 13136 and moves each needle towards its target with smooth acceleration.
- **No-data watchdog:** after 30 seconds without a packet every needle returns to its zero.

- **Zero trims:** each gauge has a `...ZeroOffset` value to nudge its true zero without touching the table.
- **Keep-alive:** `STEPPER` to the Health Monitor every 10 seconds; debug text goes to UDP 27000.

7.2 Testing a gauge on demand

Send a control packet to re-run one gauge's sweep without rebooting, for example `C,SWINGTQ:1`.

Command	Gauge
`SWINGFUEL`	Fuel quantity
`SWINGTQ`	Torque
`SWINGITT`	Turbine out temperature
`SWINGN1`	Gas producer
`SWINGVSI`	Vertical speed
`SWINGIAS`	Airspeed
`SWINGAGL`	Radar altitude

The **StepperVSITester** app can send every token and raw token to this board without the simulator. The **SwingTester** app sends these commands, and the Dual Stepper and servo sweeps, one at a time or all in a fixed order:

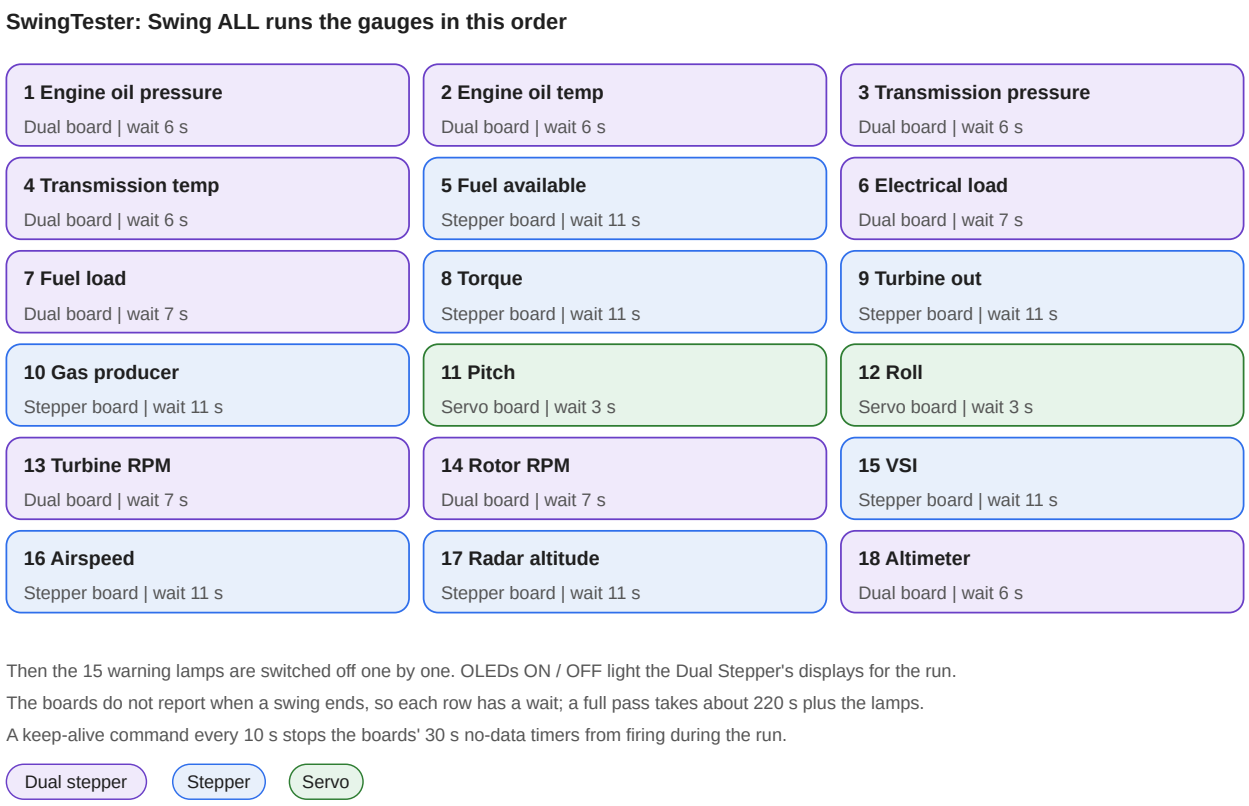


Figure 19 — Swing ALL sequence

It lights the OLEDs and the lamps during the run and sends a keep-alive so the boards' 30 second watchdogs do not fire. *Reset boards...* reboots the boards you tick (Radio, Servo, Upper, Stepper, Dual Stepper, USB HID) by sending each one `C,RESET:1`; each restarts about 2 seconds later and repeats its power-up test.

7.3 Known issues

- The backlight PWM was once on pin 8, which the PCB routes to VSI coil B. It is now pin 5, matching the PCB's `BACKLIGHT_PWM` net, in both stepper sketches, and it follows `BPWR`, `C,B:n` and the 30 second no-data watchdog (see Backlight).
- Pin 32 is also claimed by an unused roll stepper in the code.

- The oil, transmission and placeholder gauges use a shared linear scale, not bench-measured tables.

8 OLED dual stepper controller

A third Mega, on the Jet Ranger OLED Dual Stepper PCB at 172.16.1.106, drives stepper gauges plus four small OLED displays. It runs `JET_RANGER_OLED_DUAL_STEPPER_CONTROLLER.ino` and, like the 13 Stepper board, takes real units.

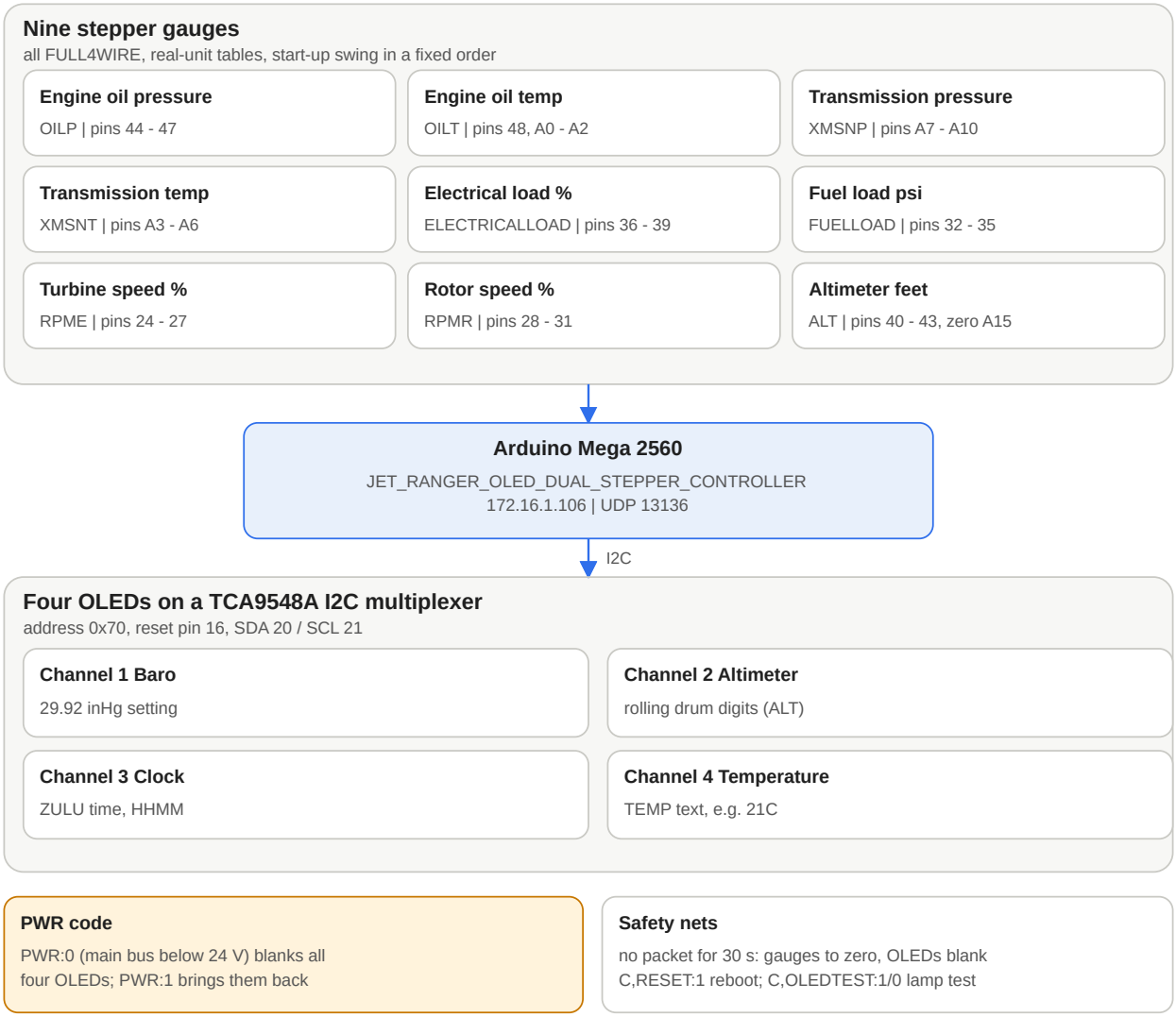


Figure 20 — Dual stepper board

Gauge	Token	Mega pins	Units	Calibration
Engine speed (N2)	`RPME`	24 to 27	percent, one decimal	4-point table
Rotor speed	`RPMR`	28 to 31	percent, one decimal	4-point table
Altimeter	`ALT`	40 to 43, zero sensor A15	feet	720 steps per 1000 ft
Fuel load	`FUELLOAD`	32 to 35	psi	2-point table (0 to 30 psi)
Electrical load	`ELECTRICALLOAD`	36 to 39	percent	2-point table
Engine oil temperature	`OILT`	48, A0 to A2	degrees C	4-point table
Engine oil pressure	`OILP`	44 to 47	psi	4-point table
Transmission oil temperature	`XMSNT`	A3 to A6	degrees C	4-point table
Transmission oil pressure	`XMSNP`	A7 to A10	psi	5-point table

Turbine out temperature (`ITT`) and fuel quantity (`FUEL`) are also accepted, as on the 13 Stepper board. Wire each physical gauge to one board only; the matching channel on the other board then simply has nothing connected. Each token also has a `RAW` twin for bench work. `IAS` is not handled here; the FSUIPC and MSFS bridges send airspeed to the 13 Stepper board with one decimal place so the needle moves smoothly.

8.1 OLED displays

The four 128 by 32 SSD1306 OLEDs share the Mega's I2C pins (20 SDA, 21 SCL) through a TCA9548A multiplexer at address 0x70. The firmware pulses the mux reset on pin 16 at start-up.

Display	Mux channel	UDP token	Shows
Barometric setting	1	none yet	`2992` at boot; not updated
Altimeter digits	2	`ALT`	Rolling-drum altitude, raw MSL (barometric correction off)
Clock	3	`ZULU`	Zulu time from HHMM, for example `ZULU:1430`
Temperature	4	`TEMP`	Text as sent, for example `TEMP:25C`

Send `C,OLEDTEST:1` to light all four displays solid and `C,OLEDTEST:0` to restore them. When the main bus is below 24 V the bridge sends `PWR:0` and all four displays go blank; `PWR:1` turns them back on and they repaint within about a second. The altimeter needle keeps following altitude. The lamp test still lights all four displays whatever `PWR` says.

8.2 Behaviour

- **Power-up:** OLEDs start first, then each gauge sweeps in turn: engine oil pressure, engine oil temperature, transmission pressure, transmission temperature, electrical load, fuel load, engine RPM, rotor RPM, altimeter. The altimeter seeks its zero sensor before sweeping. Start-up takes about 65 seconds.
- **No-data watchdog:** after 30 seconds without a packet the needles return to zero and all OLEDs blank.
- **On-demand sweeps:** `C,SWINGOILP:1`, `SWINGOILT`, `SWINGXMSNP`, `SWINGXMSNT`, `SWINGELECTRICALLOAD`, `SWINGFUELLOAD`, `SWINGRPME`, `SWINGRPMR`, `SWINGALT`. Each blocks the board for 5 to 7 seconds.
- **Keep-alive:** `DUAL_STEPPER` to the Health Monitor every 10 seconds, so it shows separately from the 13 Stepper board.

8.3 Known issues

- RAM use is 75 percent or more; avoid adding more OLEDs or string-heavy code.
- The latest bench I2C scan found OLEDs on mux channels 1 to 4; check by eye that each shows its text.
- Engine and rotor RPM tables assume 0 percent is 0 steps; `RSoffset` is not yet measured.
- The altimeter zero seek travels up to 10,080 steps (about 14 turns) before giving up.

9 Radio controller (COM and NAV)

The radio panel is another Mega at 172.16.1.101, running `JET_RANGER_RADIO_CONTROLLER.ino`. It shows the COM and NAV frequencies on two OLEDs, reads eight tuning knobs and a 4 by 4 button matrix, and sends knob and button actions back to the PC.

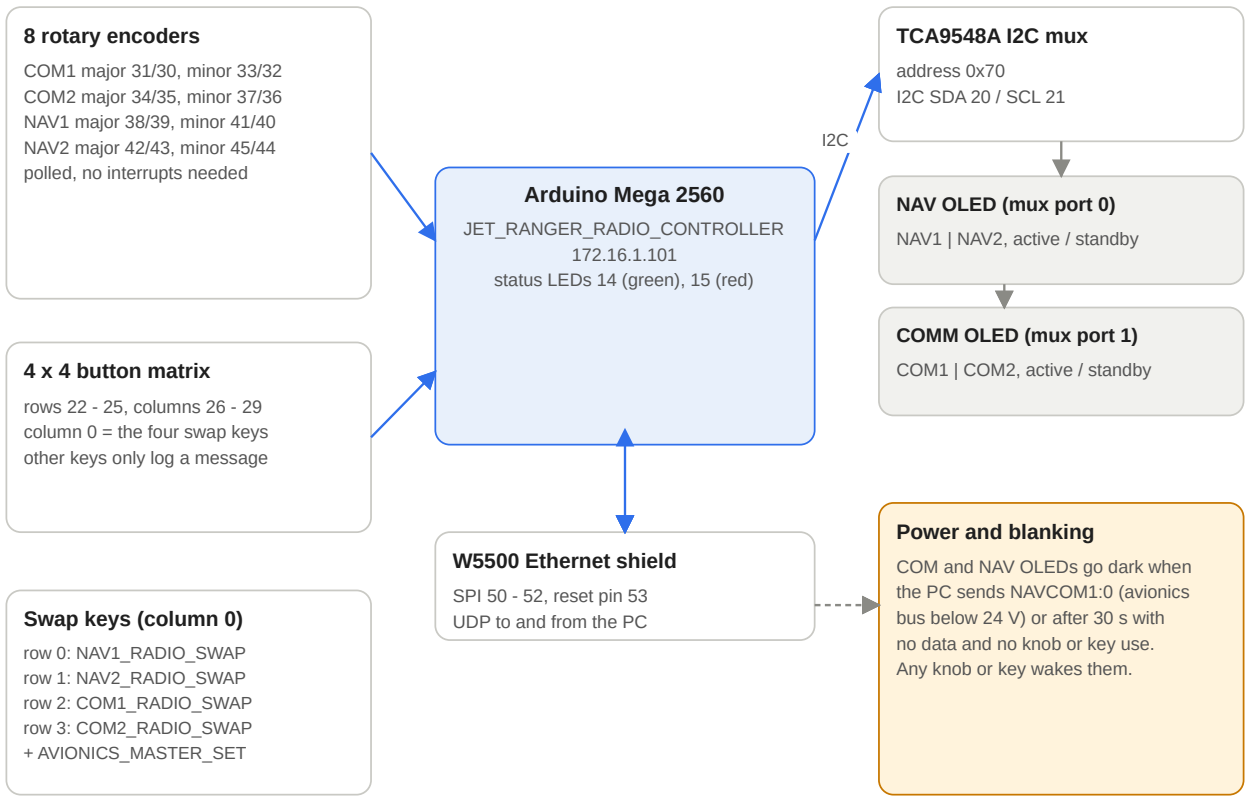


Figure 21 — Radio panel

9.1 Boards

PCB	Role	Main parts
Jet Ranger Radio Auto Pilot	Radio controller board	Mega 2560 shield, Ethernet module, TCA9548A I2C mux
Jet Ranger COM and NAV	Front panel board behind the radio faceplate	8 rotary encoders, push buttons, four frequency-swap switches

9.2 Inputs and outputs

Function	Mega pins	Notes
COM1 major and minor knob	31, 30 and 33, 32	Encoder A, B
COM2 major and minor knob	34, 35 and 37, 36	Encoder A, B
NAV1 major and minor knob	38, 39 and 41, 40	Encoder A, B
NAV2 major and minor knob	42, 43 and 45, 44	Encoder A, B
Button matrix	rows 22 to 25, columns 26 to 29	4 by 4
OLEDs	I2C through the TCA9548A (0x70)	Port 0 = NAV OLED, port 1 = COM OLED, both at 0x3C
Status LEDs	14 green, 15 red	Red flashes while Ethernet starts
Ethernet module	SPI 50 to 52, reset 53	

9.3 Data from the PC (UDP 13136)

Token	Meaning	Example
`C1A`, `C1S`	COM1 active and standby frequency	`C1A:119.500`
`C2A`, `C2S`	COM2 active and standby frequency	`C2S:121.900`
`NAVCOM1`	Radio power (0 = off)	`NAVCOM1:1`
`MAINBUS`	Main bus voltage	
`C,B:n`	Display brightness (control packet)	`C,B:5`

9.4 Commands to the PC (UDP 27001)

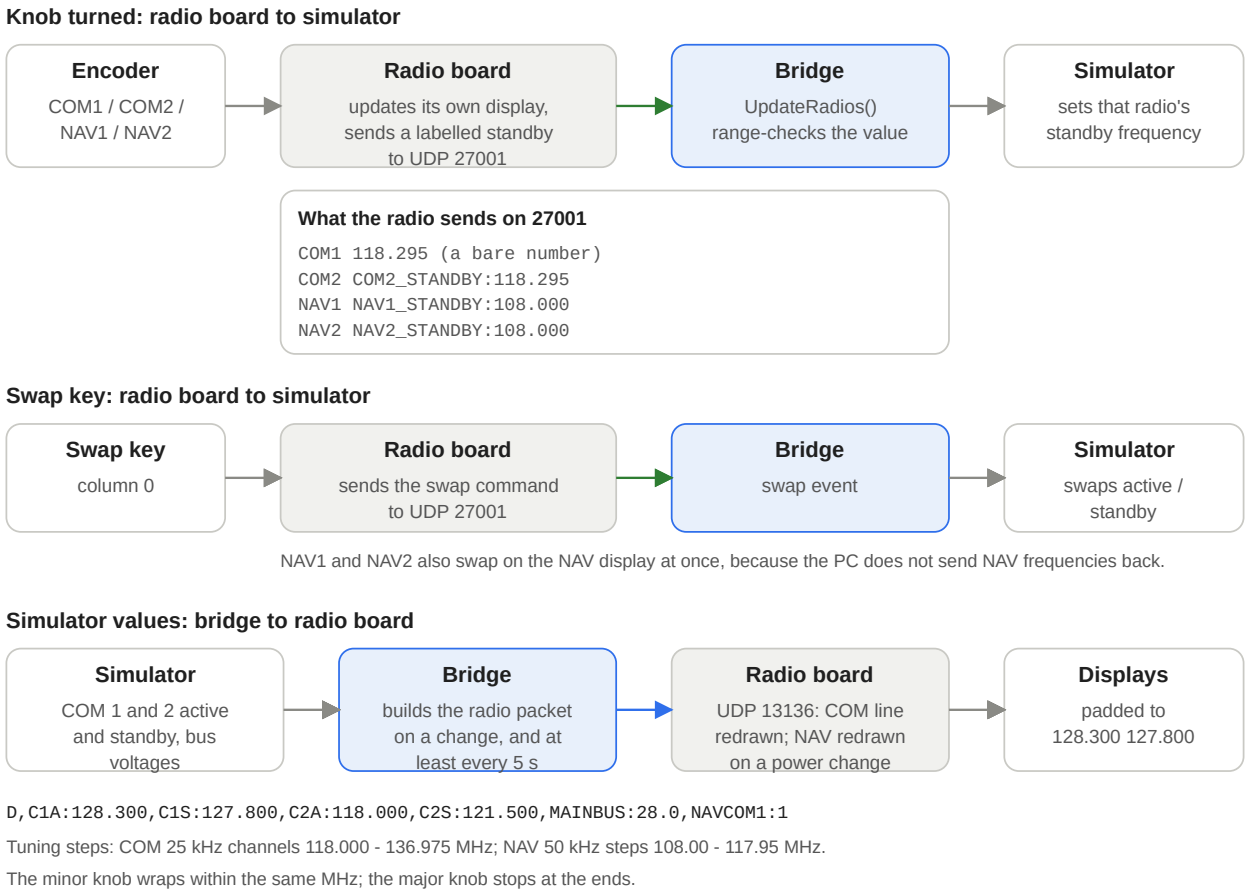


Figure 22 — Where each input goes

Turning a knob changes the standby frequency on the OLED and sends the new value to the PC. The four swap switches share matrix column 0:

Switch (row, column 0)	Command sent
Row 0: NAV1 FREQ	`NAV1_RADIO_SWAP`
Row 1: NAV2 FREQ	`NAV2_RADIO_SWAP`
Row 2: COMM1	`COM1_RADIO_SWAP`
Row 3: COMM2	`COM2_RADIO_SWAP`, then `AVIONICS_MASTER_SET`

The bridge listens on 27001 and turns these into simulator events. NAV1 and NAV2 also swap their active and standby frequencies on the NAV OLED straight away, because the PC does not send NAV frequencies back; the COM swaps change the display only when the simulator returns the swapped frequencies. Other buttons are only logged to the debug listener.

Each knob sends its standby frequency with the radio's name: COM1 as the bare number (for example `118.295`), the others as `COM2_STANDBY:118.295`, `NAV1_STANDBY:108.000` or `NAV2_STANDBY:108.000`.

Nothing is sent during the first 1.5 seconds after power-up, so the knobs' start positions do not change the simulator. Each OLED line always shows two frequencies in a fixed 16-column layout.

Tuning steps follow the standard channel spacing. The COM knobs run 118.000 to 136.975 MHz in 25 kHz channels (the major knob picks 118 to 136, the minor knob 000, 025, 050 and 075). The NAV knobs run 108.00 to 117.95 MHz in 50 kHz steps (major 108 to 117, minor 000, 050, 100 and so on). Each minor knob wraps round at the ends. 8.33 kHz channels are not offered. FSUIPC holds COM frequencies to 10 kHz, so in that simulator a channel such as 118.025 may appear as 118.02 or 118.03.

9.5 Display behaviour

- If no data arrives from the PC for 30 seconds and no knob or button is used, both OLEDs blank (as if the radio had no power). Turning a knob, pressing a button or new data switches them back on.
- When the avionics bus is below 24 V the bridge tells the radio the power is off (`NAVCOM1:0`) and the COM and NAV displays go blank together; they light together again when the bus rises.
- At power-up the matrix is ignored for the first 1.5 seconds so a floating input cannot send false button presses.
- The radio sends `COMM_NAV` to the Health Monitor every 10 seconds.

9.6 Known issues

- Only COM1 and COM2 frequencies arrive from the PC; there are no NAV frequency tokens yet, although the NAV knobs are read.
- The Radio Auto Pilot PCB has a backlight driver on D5, autopilot knobs, volume pots and display drivers that the radio sketch does not use yet, and `C,B:n` brightness commands are accepted but have no effect.
- Four encoders (COM1 major and minor, COM2 minor, NAV1 minor) have A and B the other way round from the PCB labels; it only matters if one of those knobs tunes backwards.
- If every button in the matrix's first column stops working, send `C,MATRIX:1` to port 13136: the radio prints the raw H or L state of each column to the debug listener. A column that reads L with nothing pressed is a hardware fault on that Mega pin or board (a faulty or badly seated Mega did exactly this on the bench).

10 Joystick (HID) interface

Overhead and console switches reach the simulator as ordinary USB joystick buttons. The upper controller scans the switches and sends each change over UDP to a small USB board, `UDP_TO_HID`, which Windows sees as four joysticks and a keyboard. The joysticks have buttons only: no axes and no hat switches. Bind them like any other controller, in FSUIPC for Prepar3D and FSX or in the MSFS controls menu.

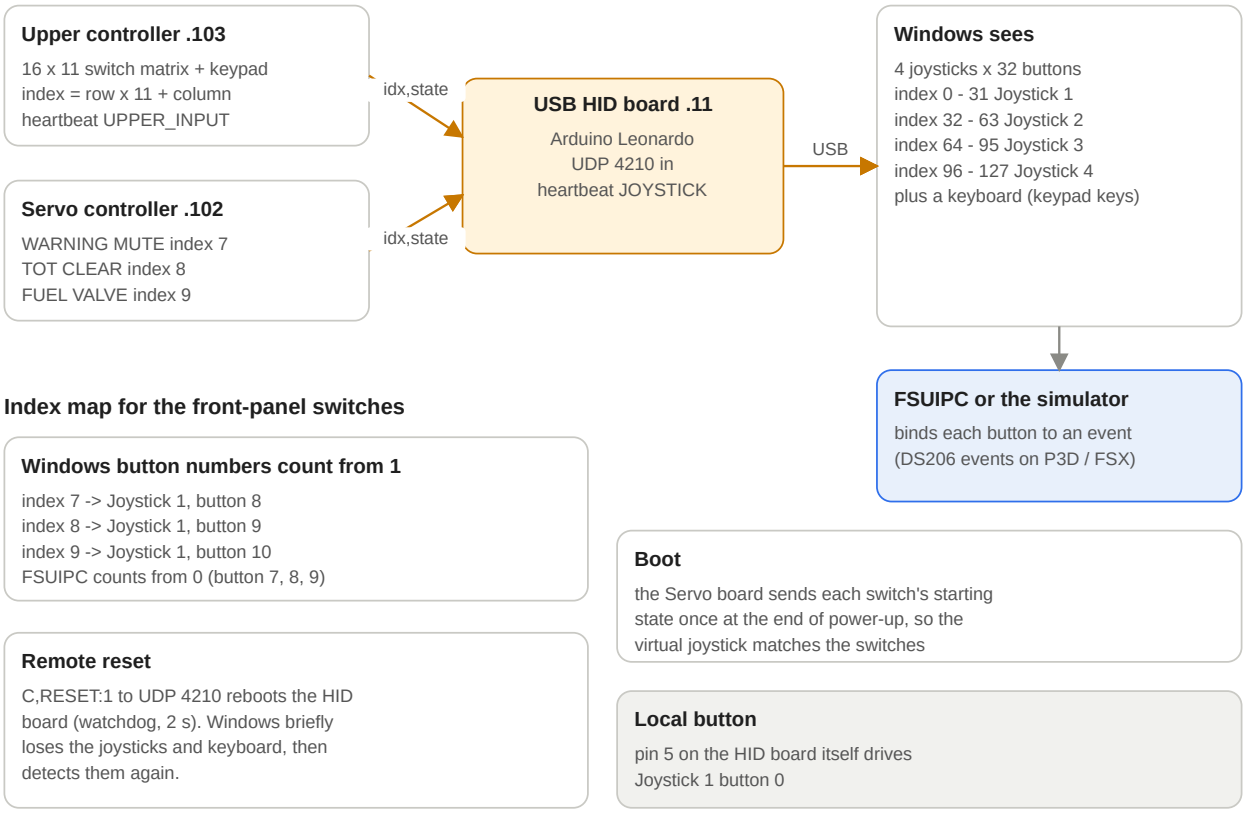


Figure 23 — From switch to joystick button

Board	Sketch	Address	Role
Upper controller (Mega 2560)	`JET_RANGER_UPPER_CONTROLLER.ino`	172.16.1.103	Scans a 16 by 11 switch matrix (rows on pins 22 to 37, columns on 38 to 48) and a 4 by 3 keypad (A9 to A15)
USB HID board (Leonardo class)	`UDP_TO_HID.ino`	172.16.1.11	Listens on UDP 4210 and presents 4 joysticks of 32 buttons plus a keyboard
Servo board (Mega 2560)	`JET_RANGER_SERVO_CONTROLLER.ino`	172.16.1.102	Also sends three front-panel switches (warning mute, TOT clear, fuel valve) as index 7, 8 and 9

10.1 How a switch becomes a joystick button

1. The upper controller numbers each matrix position as `index = row x 11 + column` (0 to 175).
2. When a switch changes it sends `index,state` to `172.16.1.11:4210`, with state 1 = switch on and 0 = off.
3. The HID board picks joystick = index div 32 and button = index mod 32, then holds that button down while the switch is on.
4. Keypad keys are sent as `KEY,<key>,<state>` and typed as keyboard keys 0 to 9, `*` and `#`.
5. The servo board sends the same format for its three switches: index 7 (warning mute, pin 47), 8 (TOT clear, pin 43) and 9 (fuel valve, pin 48), with 1 = closed.

Index range	Windows device
0 to 31	Joystick 1
32 to 63	Joystick 2
64 to 95	Joystick 3
96 to 127	Joystick 4
128 to 175	Not sent to Windows (out of range)

10.2 Switch mapping in use

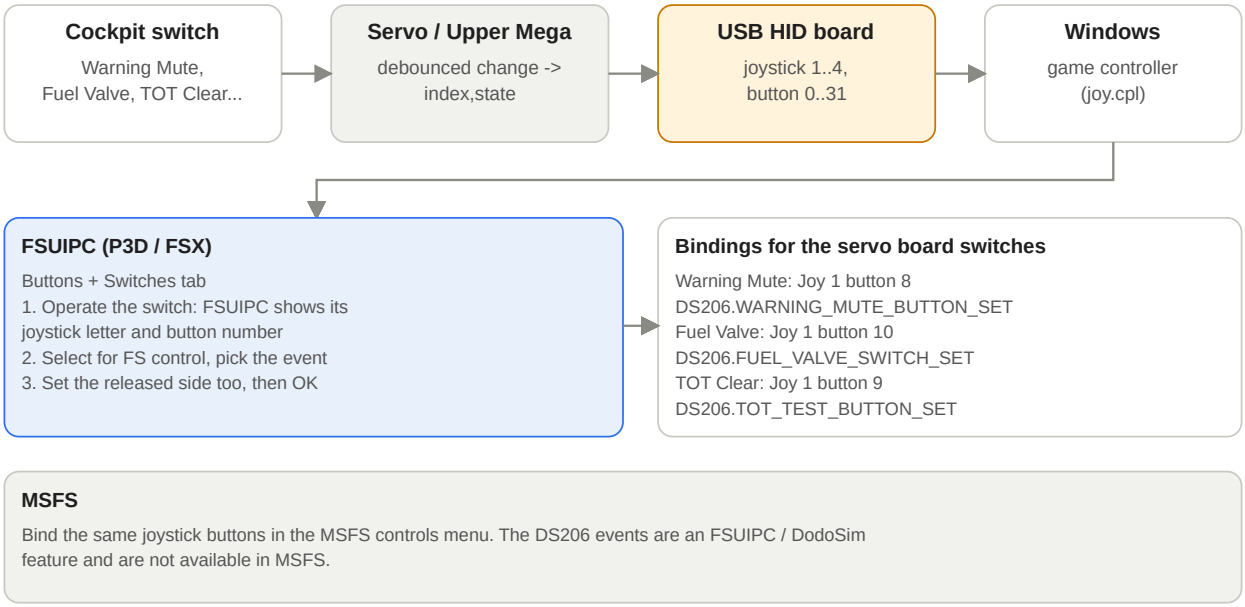
From `UpperSwitchMapping.ods`. Button numbers are as Windows and the simulators show them (1 to 32); the firmware counts from 0. A latching switch holds its button down while on, so bind both press and release: in FSUIPC use the `_SET` event with parameter 1 on press and 0 on release.

Index	Source	Switch	Joystick, button	Suggested DodoSim event
0 to 4	Upper 0, 0-4	Heater rotary HTR 0 to 4	J1, 1 to 5	
5	Upper 0, 5	Air vent	J1, 6	`HEAT_VENT_SWITCH_SET`
6	Upper 0, 6	Air heat	J1, 7	`HEAT_VENT_SWITCH_SET`
7	Servo D47	Warning mute	J1, 8	`DS206.WARNING_MUTE_BUTTON_SET`
8	Servo D43	TOT clear (TOT light test)	J1, 9	`DS206.TOT_TEST_BUTTON_SET`
9	Servo D48	Fuel valve	J1, 10	`DS206.FUEL_VALVE_SWITCH_SET`
11	Upper 1, 0	Forward fuel boost breaker	J1, 12	`FORE_FUEL_PUMP_CB_SET`
12	Upper 1, 1	Aft fuel boost breaker	J1, 13	`AFT_FUEL_PUMP_CB_SET`
13	Upper 1, 2	Generator field breaker	J1, 14	`GENERATOR_FIELD_CB_SET`
14	Upper 1, 3	Generator breaker	J1, 15	`GENERATOR_RESET_CB_SET`
15	Upper 1, 4	Directional gyro / attitude indicator	J1, 16	`DIR_GYRO_SWITCH_SET`
16	Upper 1, 5	Battery	J1, 17	`BATTERY_SWITCH_SET`
17	Upper 1, 6	Generator (on / start)	J1, 18	`GENERATOR_SWITCH_SET`
22	Upper 2, 0	Landing light	J1, 23	`LANDING_LIGHT_SWITCH_SET`
23	Upper 2, 1	Instrument lights	J1, 24	`PANEL_LIGHTS_SWITCH_SET`
24	Upper 2, 2	Caution light breaker	J1, 25	`CAUTION_LT_CB_SET`
25	Upper 2, 3	Anti-collision light	J1, 26	`COLLISION_LIGHTS_SET`
26	Upper 2, 4	Position lights	J1, 27	`POSITION_LIGHTS_SET`
27	Upper 2, 5	Pitot heat	J1, 28	`PITOT_HEAT_SWITCH_SET`
28	Upper 2, 6	Avionics	J1, 29	`AVIONICS_SWITCH_SET`
39	Upper 3, 6	Strobe light	J2, 8	none yet

The suggested events are the closest match in `DODSIM206V2.EVT`; the bindings actually saved in FSUIPC or MSFS are not in the repository. All other matrix positions are wired but unassigned.

10.3 Setting FSUIPC events from the joystick buttons

A joystick button does nothing in the simulator until it is bound to an event in FSUIPC. The **Warning Mute** and **Fuel Valve** switches on the servo board must be set this way before they work; the **TOT clear** switch beside them is bound the same way. This applies to Prepar3D and FSX with FSUIPC and `DODSIM206V2.EVT` installed. In MSFS, bind the buttons in the controls menu instead; the DS206 events are not available there.



FSUIPC counts buttons from 0 and Windows from 1, so Windows button 10 appears as button 9 in FSUIPC.

Figure 24 — Binding a switch

1. Start the simulator with the USB HID board connected. Windows should list the four joysticks (Game Controllers, `joy.cpl`); pressing a switch lights the matching button there.
2. In the simulator open FSUIPC from the Add-ons menu and choose the **Buttons + Switches** tab.
3. Operate the switch you want to bind (for example flip the Fuel Valve switch). FSUIPC shows the joystick letter and button number it saw. FSUIPC counts buttons from 0 and Windows from 1, so Windows button 10 shows as button 9 in FSUIPC.
4. Under the pressed (or closed) side, tick *Select for FS control*, choose the event from the list and type the parameter in the box beside it. The DS206 events are listed alphabetically.
5. A switch that stays on needs the released side set as well: tick *Select for FS control* there, choose the same event and enter the opposite parameter, so the cockpit follows the switch in both directions.
6. Click OK, flip the switch and check the result in the cockpit. After an event file update FSUIPC re-indexes the events alphabetically, so check these bindings again.

- **Warning Mute:** Joystick 1 button 8 (FSUIPC button 7). Event `DS206.WARNING_MUTE_BUTTON_SET`, parameter 1 when pressed and 0 when released. Alternatively bind `DS206.WARNING_MUTE_BUTTON_TRIGGER` (no parameter) to the pressed side only.
- **Fuel Valve:** Joystick 1 button 10 (FSUIPC button 9). Event `DS206.FUEL_VALVE_SWITCH_SET`, parameter 1 when closed and 0 when released. The valve in the simulator follows the switch; if it ends up the wrong way round, swap the two parameters.
- **TOT clear (TOT light test):** Joystick 1 button 9 (FSUIPC button 8). Event `DS206.TOT_TEST_BUTTON_SET`, parameter 1 when pressed and 0 when released.

If FSUIPC does not see a switch, check that Windows shows the button moving in `joy.cpl` first. If it does not, the problem is on the board side: the servo board's debug messages report each change and the HID board logs the button it sets.

10.4 Known issues

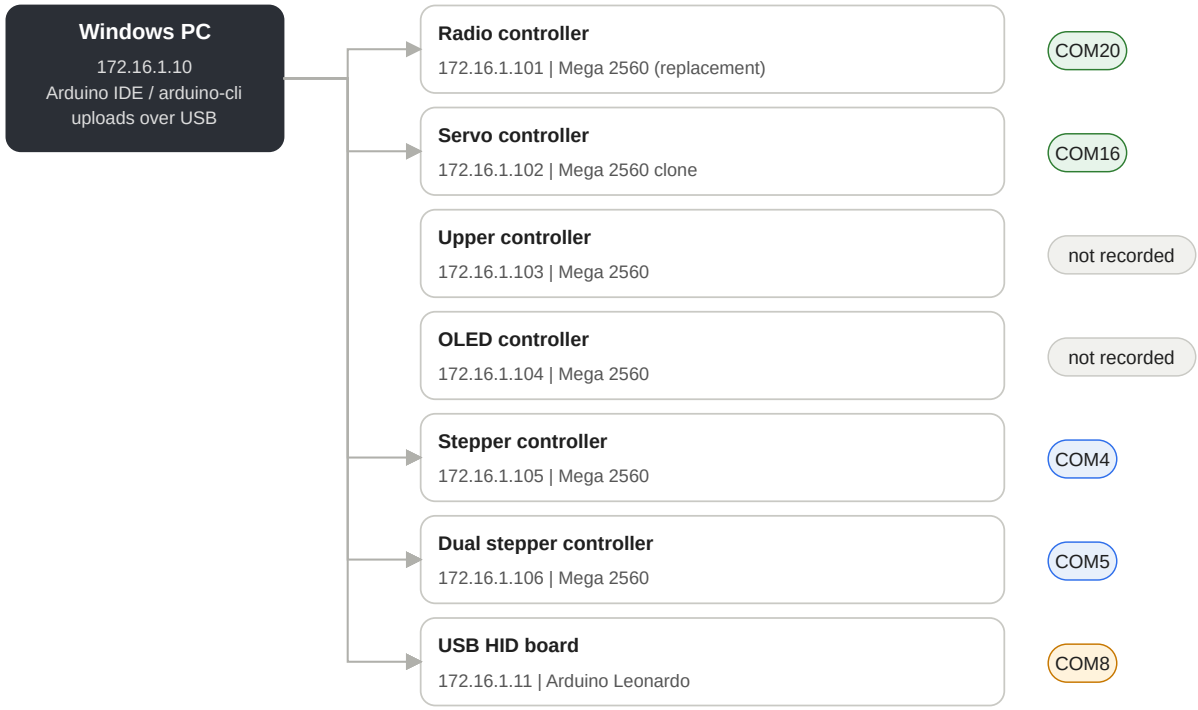
- The HID board's own test button on pin 5 also drives Joystick 1 button 1, the same button as HTR 0.
- `CreateDcsBiosMessage()` is still a stub, so these switches do nothing in DCS; only the joystick path works.
- The HID board reboots when it receives `C,RESET:1` (or `RESET`) on UDP 4210. Its USB connection drops while it restarts, so Windows briefly loses the four joysticks and the keyboard; re-check any game that has them open.

- The servo board's three switches use indexes 7, 8 and 9, which Windows shows as Joystick 1 buttons 8, 9 and 10. If they were meant to be Windows buttons 7, 8 and 9, button 7 is already the Air heat switch (index 6), so the indexes would have to move.

11 COM ports and flashing

Each Arduino is programmed over its own USB cable from the primary PC. The cockpit itself runs over Ethernet, so USB is only needed to upload a sketch.

11.1 COM port to Arduino mapping



COM numbers are the ones used at the last flash. They follow the USB socket and the board, so check them (see the guide) before an upload. Each board also has a network connection: USB is for programming only, the cockpit runs over UDP.

Figure 25 — Which board is on which COM port

Board	Address	Hardware	COM port	Sketch
Radio controller	172.16.1.101	Arduino Mega 2560 (replacement board)	**COM20**	`JET_RANGER_RADIO_CONTROLLER`
Servo controller	172.16.1.102	Mega 2560 clone	**COM16**	`JET_RANGER_SERVO_CONTROLLER`
Upper controller	172.16.1.103	Mega 2560	not recorded	`JET_RANGER_UPPER_CONTROLLER`
OLED controller	172.16.1.104	Mega 2560	not recorded	`JET_RANGER_OLED_CONTROLLER`
Stepper controller	172.16.1.105	Mega 2560	**COM4**	`JET_RANGER_STEPPER_CONTROLLER`
Dual stepper controller	172.16.1.106	Mega 2560	**COM5**	`JET_RANGER_OLED_DUAL_STEPPER_CONTROLLER`
USB HID board	172.16.1.11	Arduino Leonardo	**COM8**	`UDP_TO_HID`

These are the ports used the last time each board was flashed. A COM number belongs to a USB socket and a board together, so it **can change**:

- The Servo controller is a clone Mega that reports no USB serial number, so Windows ties its COM number to the USB socket it is plugged into. Plug it into a different socket and it appears as a new port.
- Replacing a Mega (the Radio controller was replaced once) gives a new port number.
- The Upper and OLED controllers were not connected when the ports were recorded.
- The Lowrance GPS cable is separate: the GPS interface asks for its serial port the first time it runs and remembers it.

Always check the port before an upload: open Device Manager, Ports (COM and LPT), or run `arduino-cli board list`. If you are not sure which board is which, unplug one USB cable and see which port disappears. Add the confirmed port to the table above (and to the sketch header comment) once you have uploaded to a board.

11.2 Flashing a board

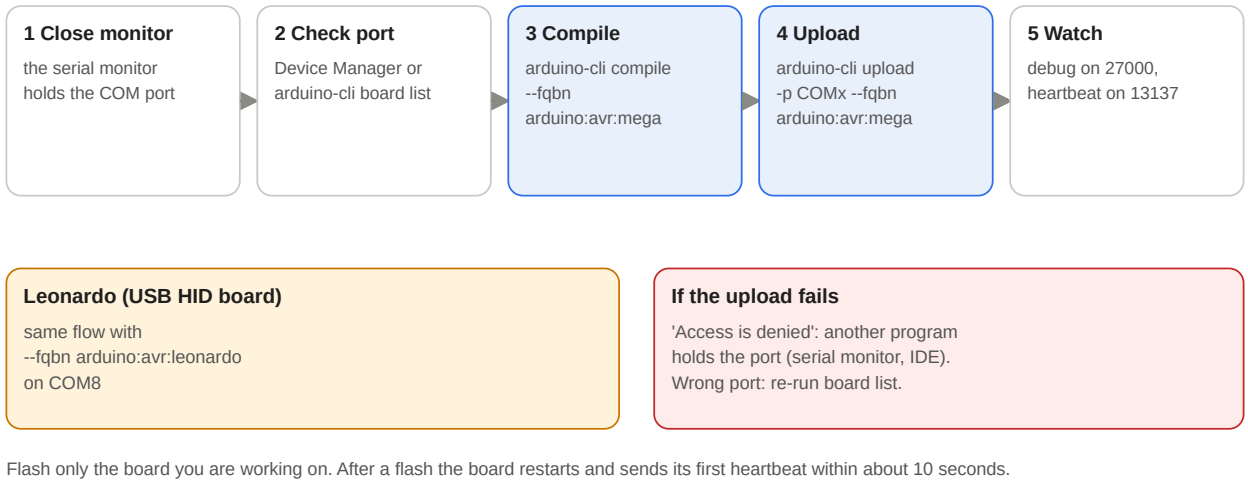


Figure 26 — Flashing a board

1. Close the Arduino IDE **serial monitor**. It holds the port, and an upload then fails with *Access is denied*.
2. Check the COM port (above).
3. Compile the sketch.
4. Upload to that port.
5. Watch the debug listener (UDP 27000) and the Health Monitor: the board prints *Ethernet Started* and *Setup Complete* and its light goes green within about 10 seconds.

```
arduino-cli compile --fqbn arduino:avr:mega "Jet Ranger Arduino
Sketches\JET_RANGER_STEPPER_CONTROLLER"
arduino-cli upload -p COM4 --fqbn arduino:avr:mega "Jet Ranger Arduino
Sketches\JET_RANGER_STEPPER_CONTROLLER"
```

For the Leonardo (USB HID board) use `--fqbn arduino:avr:leonardo` and its port, COM8. Flash only the board you are working on. If a board ever reboot-loops after a reset packet, power-cycle it and flash it again.

12 PCBs and wiring

The Mega sits on a carrier PCB that breaks its pins out to servo headers, lamp outputs, gauge sockets and the Ethernet module. KiCad projects for every board are in `Jet Ranger/PCBs` in the repository, with Gerbers in each `MANUFACTURING` folder. The list below was brought up to date on 18 October 2026.

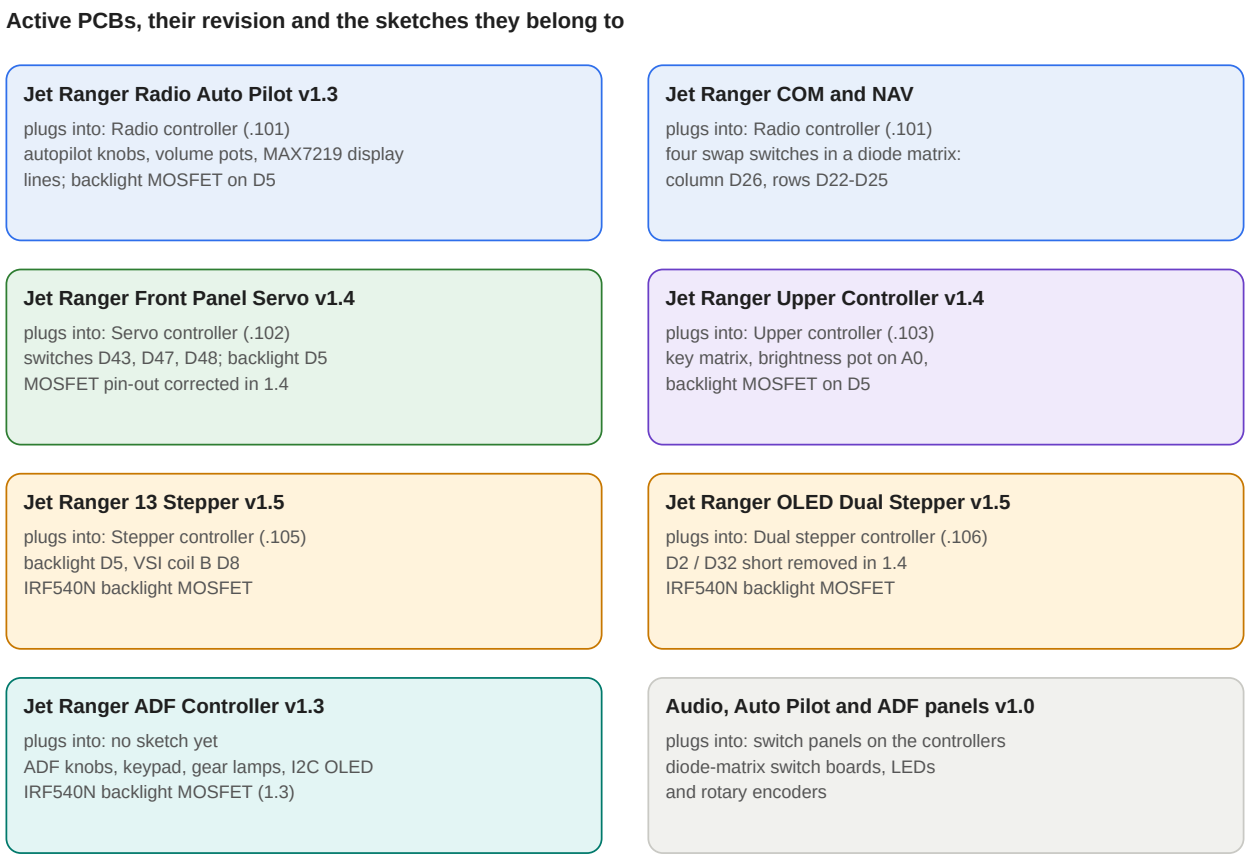


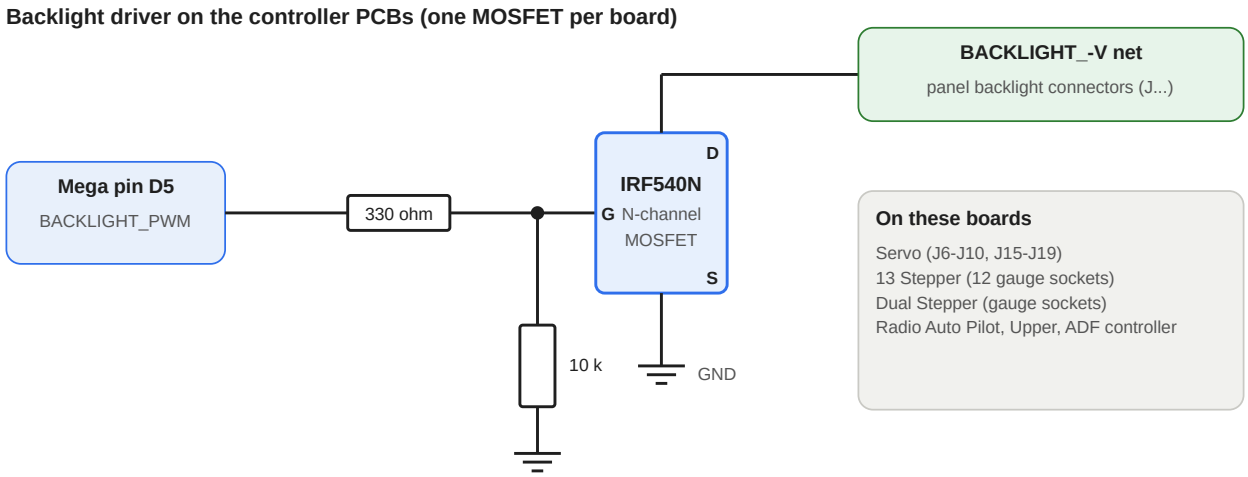
Figure 27 — Active PCBs, their revision and their sketches

PCB	Revision	Used by	Main parts
Jet Ranger Front Panel Servo	1.4	Servo board	Mega shield, ULN2803A lamp drivers, servo headers, lamp outputs, three switch inputs, backlight MOSFET
Jet Ranger 13 Stepper	1.5	Stepper board	Mega shield, USR-ES1 Ethernet, 14 six-pin gauge sockets, 12 V lamp supply with polyfuses, backlight MOSFET
Jet Ranger OLED Dual Stepper	1.5	Dual stepper board	Mega shield, USR-ES1, six-pin gauge sockets, TCA9548A sockets for the OLEDs, backlight MOSFET
Jet Ranger X27 Small Stepper	not set	Each stepper gauge	One X27 or VID29 stepper, backlight LED, six-pin connector to the controller
Jet Ranger Radio Auto Pilot	1.3	Radio controller	Radio and autopilot controller with the backlight MOSFET
Jet Ranger COM and NAV	not set	Radio panel	8 rotary encoders, push buttons, four frequency-swap switches
Jet Ranger Upper Controller	1.4	Upper controller	Mega shield for the switch matrix, backlight MOSFET, brightness input

PCB	Revision	Used by	Main parts
Jet Ranger ADF Controller	1.3	ADF controller (no sketch yet)	ADF knobs, keypad, gear lamps, I2C OLED, backlight MOSFET
Jet Ranger Audio Panel, Auto Pilot Panel, ADF Panel	1.0	Switch panels	Diode-matrix switch boards with LEDs and rotary encoders

The X27 steppers run straight from the Mega's pins, so the stepper boards need no driver chips; the 12 V input feeds lamps and backlights.

12.1 Backlight MOSFET (revisions of 18 October 2026)



Revisions 1.3 to 1.5 (18 October 2026) replaced the generic MOSFET symbol, which had source and drain swapped, with the IRF540N.
A board built from an earlier revision has the transistor pinned wrongly: check it, or fit it the right way round.

Figure 28 — The backlight driver

Every controller board switches its panel backlights with one **IRF540N** MOSFET: Mega pin D5 (`BACKLIGHT_PWM`) goes through 330 ohm to the gate, a 10 k resistor holds the gate to ground, the drain is the shared backlight net and the source is ground. The earlier revisions used a generic MOSFET symbol with **source and drain swapped**, so the 1.4 Servo, 1.3 Radio Auto Pilot, 1.4 Upper, 1.3 ADF Controller and 1.5 Stepper and Dual Stepper boards were revised to use the IRF540N with the correct pin-out. If you have a board from an earlier revision, check the transistor before relying on the backlight.

Version 1.4 of the OLED Dual Stepper PCB (12 October 2026) separated Mega pins 2 and 32. Earlier versions joined them in copper because of a copy-and-paste net name, so the fuel quantity and fuel load gauges shared coil A. On a board older than 1.4 fit only one of those two gauges or cut the trace between the pins.

Lamp pins on the Servo PCB. Ten of the 15 lamp outputs are on different Mega pins from the sketch (see Known issues); test each lamp on its own.

Radio Auto Pilot knobs. The PCB also carries the autopilot knobs, volume pots, display drivers and a fifth matrix column that the radio sketch does not read yet, and four of its encoders have A and B swapped relative to the sketch.

12.2 Connections on the servo carrier board

Connector	Count	Pins	Wiring
Servo headers	16 on the PCB, 2 used	26 (pitch), 27 (bank)	Signal, +V, GND; +V from the servo rail
Lamp outputs	15	A1 to A15	Pin HIGH = lamp on; through a driver for lamps above 20 mA
Spare outputs	3	not assigned	Reserved for future lights
Ethernet module	1	SPI 50 to 52, chip select, reset 53	Reset is pulsed by the firmware at start-up

Connector	Count	Pins	Wiring
Status LEDs	2	14 green, 15 red	With series resistors to GND
Servo power input	1		5 to 6 V; ground joined to Mega GND
Front-panel switch inputs	3	D43, D47, D48	Each to GND through a switch (SW1 TOT clear, SW3 warning mute, SW2 fuel valve); internal pull-up, debounced in firmware

The rule that matters most: **never power servos from the Mega's 5 V pin.** Servos draw current spikes when they move, which can reset the Mega and drop the Ethernet link.

12.3 Wiring checklist

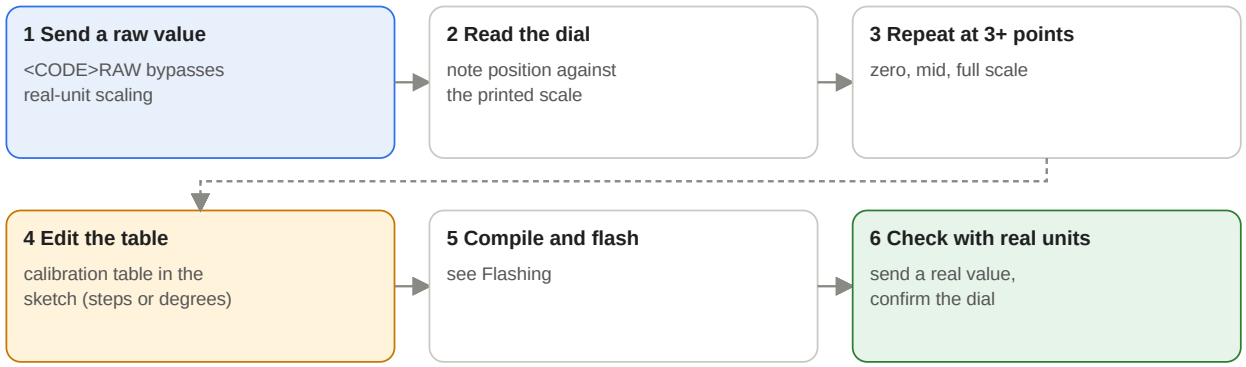
- Servo rail fed from its own 5 to 6 V supply, not the Mega.
- Supply ground joined to Mega GND (signals need a shared reference).
- Each servo plugged in signal-side matching the header silkscreen.
- Lamps checked during the power-up lamp test.
- Ethernet cable to the same switch as the primary PC.
- Board serial number noted in the sketch header.

13 Calibration

The servo board turns attitude degrees into a servo position (0 to 180) with two small tables in the sketch. The stepper boards do the same for steps. Between table points the boards interpolate in a straight line; beyond the ends they hold the end value.

Instrument	Attitude	Servo position
Pitch (`PITCH_DEG_TABLE`)	-30 degrees	179
Pitch	0 (level)	113
Pitch	+30 degrees	70
Bank (`BANK_DEG_TABLE`)	-90 degrees	5
Bank	0 (level)	93
Bank	+90 degrees	179

The self-test sweep uses `aServMinPosition[]` and `aServMaxPosition[]` (bank 5 to 179, pitch 166 to 70) and parks at `aServZeroPosition[]` (bank 93, pitch 113).



Typical tables: pitch -30 / 0 / +30 degrees, bank -90 / 0 / +90 degrees, one row per stepper or servo gauge.

Sending <CODE>RAW:n moves the motor to step n directly, so any dial can be calibrated without a simulator.

Figure 29 — The calibration loop

13.1 Calibrating a servo

1. Send raw positions with `BANKRAW` or `PITCHRAW` (for example `D,PITCHRAW:113`) until the instrument shows a known attitude.
2. Note the position for each attitude you check: at least both ends and level.
3. Enter them in `PITCH_DEG_TABLE` or `BANK_DEG_TABLE`, and set `aServZeroPosition[]` to the level position so the self-test parks there.
4. Upload, then check with real values, for example `D,PITCH:0` and `D,BANK:30`.

13.2 Calibrating a stepper gauge

1. Send raw steps with the `RAW` token (for example `D,TQRAW:300`) and read the dial.
2. Repeat at zero, mid scale and full scale (more points where the scale is not linear).
3. Edit the gauge's table in the sketch (`TQ_PCT_TABLE`, `IAS_KT_TABLE` and so on) and its `...ZeroOffset` trim if the zero is off.
4. Upload and check with real values (`D,TQ:60`).

The StepperVSITester sends every token and raw token without a simulator. Do not drive a gauge hard against its mechanical stop: a stalled servo or stepper draws high current and gets hot.

14 Troubleshooting

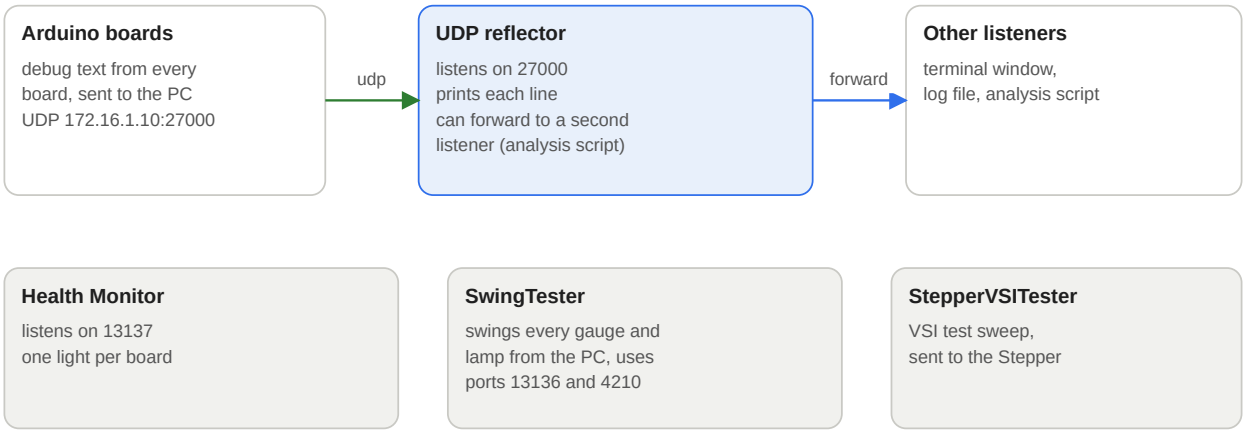


Figure 30 — Debug and test tools

Symptom	Likely cause	What to check
Red LED keeps fast-flashing, no gauge sweep	Board stuck in Ethernet start-up	Ethernet module seated; reset line on pin 53; module powered
Sweep runs but gauges never move in flight	Packets not reaching UDP 13136	PC on 172.16.1.x; Windows firewall allows outbound UDP; the bridge sends to the right boards (servo .102, steppers .105 and .106); only one bridge running
No debug messages, or the Health Monitor shows a board as down	Listener or Health Monitor not running, or PC address wrong	PC at 172.16.1.10; listener bound to UDP 27000, Health Monitor to UDP 13137; firewall allows inbound UDP on both
One gauge pinned at an end stop	Value outside the calibration table, or table points wrong	Send the `RAW` token to find the true limits, then re-measure the table (see Calibration)
All gauges drop to zero after 30 seconds	No packets arriving (watchdog)	Bridge running and connected; simulator not paused; network link
Attitude indicator lags fast changes	1 degree per 5 ms step limit	Expected; reduce `servoCheckInterval` to speed up
Servo buzzes or jitters	Weak servo supply	Separate 5 to 6 V supply for servos, common ground with the Mega
A backlight is off	Main bus below 20 V (`BPWR:0`), no packets for 30 seconds, or brightness set to 0	Check the main bus voltage in the bridge window; check the bridge is running; send a brightness with the SwingTester slider
A backlight stays on at power-up with no PC	Old firmware without the no-data rule, or the board has never been flashed with the backlight code	Flash the current Servo, Stepper and Dual Stepper sketches; the backlight should go out about 30 seconds after power-up
The backlight is always on or flickers	MOSFET pinned wrongly on a board from an earlier revision (source and drain swapped)	Check the transistor against the revision notes (PCBs and wiring)
The GPS shows Brisbane Airport	No aircraft data for 30 seconds	Check a bridge or the bench tester is running on the same PC and sending to UDP 27002; check no other program owns the port
The GPS shows the PC time, not the simulator time	The packets carry no `UTC:` (for example the FSUIPC time read failed)	The window says *PC clock*; check the bridge version
The GPS interface cannot open its port	Wrong COM port, or another program holds it	Use *Change serial port...*; close other serial programs

Symptom	Likely cause	What to check
A lamp never lights	Wiring or the lamp's driver	It should light at the power-up lamp test; if not, check the A pin and its driver
A radio knob tunes the wrong radio	Bare frequency sent instead of a labelled one	Use the real board (it labels COM2, NAV1 and NAV2); the Radio emulator sends bare numbers, so only COM1 tunes
COM and NAV displays blank	Avionics bus below 24 V, or no data and no knob use for 30 seconds	Switch avionics on; turn a knob to wake them
Dual stepper OLEDs blank	Main bus below 24 V (`PWR:0`)	Switch the battery on; check the main bus voltage in the bridge window
Radio swap switches do nothing	Column 0 of the matrix stuck	Send `C,MATRIX:1` and read the debug listener (see Radio controller)
FSUIPC binding fires the wrong event	EVT re-indexed after an update	Re-map the binding in FSUIPC
Joystick buttons vanish after a reset	USB HID board rebooted	Windows re-detects them within seconds; re-check games that had them open
Upload fails with *Access is denied*	Another program holds the COM port	Close the serial monitor and the Arduino IDE port; check the port with `arduino-cli board list`
Upload goes to the wrong board	COM port moved	Re-check Device Manager; see COM ports and flashing
Data reaches the boards only in bursts, with gaps of several seconds	The PC's Ethernet link to the board switch is dropping in and out	In Windows, check the network adapter stays connected; check the 5 V supply of the network switch stays above 4.75 V (it once dipped below and the switch dropped its ports), then the cable and switch port

14.1 Known issues in the code

Servo firmware (`JET_RANGER_SERVO_CONTROLLER.ino`)

- `powerAvailable` is set from `NAVCOM1` but nothing uses it, so the attitude indicator stays live when the simulator's power is off (the bridge sends all lamps off below 20 V).
- Several lamps log *Transmission Temperature changed* to the debug listener, whichever lamp changed.
- The PCB labels 10 of the 15 lamp outputs on different pins from the sketch (A1/A2, A5/A6, A7 to A10, A11/A12). Test each lamp on its own, for example `D,RLOW:1`, and correct whichever side is wrong.

FSUIPC event file (`DODSIM206V2.EVT`)

- The line before event 133 is missing its `//`, so FSUIPC may reject or mis-read it.
- Events 189 and 190 are named the wrong way round: 189 is called `_SET` but toggles, 190 is called `_TOGGLE` but takes 1 or 2.
- Events 69 to 71 mix up the instrument and cockpit light breakers: 69 is named `COCKPIT_LT_CB_TOGGLE`, and `INST_LT_CB_SET` appears twice.

14.2 Open questions

- Power supply rating for the servo rail and the 12 V lamp supply.
- Whether the estimates for the FlyInside and CowaSim aircraft hold at other power settings.

15 Reference: UDP tokens

From the current sketches. Gauge tokens take real units; lamp tokens take 0 (off) or 1 (on). `UDP_Interface_Values.xlsx` still lists the old 16-servo pin plan, so use these tables instead.

15.1 Gauge tokens

Token	Gauge	Board	Units
`BANK`	Attitude bank	Servo (.102)	degrees, plus and minus 90
`PITCH`	Attitude pitch	Servo (.102)	degrees, plus and minus 30
`IAS`	Airspeed	Stepper (.105)	knots, one decimal allowed
`VSI`	Vertical speed	Stepper (.105)	ft/min
`AGL`	Radar altitude	Stepper (.105)	feet
`TQ`	Engine torque	Stepper (.105)	%
`N1`	Gas producer	Stepper (.105)	%
`ITT`	Turbine out temperature	Stepper (.105) or Dual (.106)	degrees C
`FUEL`	Fuel quantity	Stepper (.105) or Dual (.106)	US gal
`OILT`	Engine oil temperature	Dual (.106) or Stepper	degrees C
`OILP`	Engine oil pressure	Dual (.106) or Stepper	psi
`XMSNT`	Transmission oil temperature	Dual (.106) or Stepper	degrees C
`XMSNP`	Transmission oil pressure	Dual (.106) or Stepper	psi
`RPME`	Engine RPM (N2)	Dual (.106)	%, one decimal
`RPMR`	Rotor RPM	Dual (.106)	%, one decimal
`ALT`	Altimeter	Dual (.106), OLED (.104)	feet
`FUELLOAD`	Fuel load	Dual (.106)	psi
`ELECTRICALLOAD`	Electrical load	Dual (.106)	%
`BARO`	Barometric setting	OLED (.104)	inHg x 100

Every gauge token also has a `RAW` twin (for example `PITCHRAW`, `IASRAW`) that takes a raw servo position or stepper step count and skips the calibration table.

15.2 Lamp tokens (servo board)

Token	Lamp	Pin
`EOUT`	Engine out	A1
`RLOW`	Rotor RPM low	A2
`TOPW`	Transmission oil pressure	A3
`TOTW`	Transmission oil temperature	A4
`BHOT`	Battery hot	A5
`BTMP`	Battery temperature	A6
`EC`	Engine chip	A7
`TRC`	Tail rotor chip	A8
`TC`	Transmission chip	A9
`BD`	Baggage door	A10
`FFLTR`	Aft fuel filter	A11
`FPMP`	Fuel pump	A12
`GENF`	Generator fail	A13
`LOWF`	Low fuel	A14
`SCF`	SC fail (standby compass)	A15

15.3 Aircraft information to the GPS interface (UDP 27002)

`D,LAT:<deg>,LON:<deg>,ALT:<feet>,IAS:<knots>,HDG:<degrees true>,UTC:<yyyyMMddHHmmss[.fff]>`, sent every 500 ms by a bridge or the bench tester. Any subset of

keys works, and the CSV form `lat,lon,altitude_ft,airspeed_kt,heading_deg[,utc]` is accepted too. South and west are negative.

15.4 Status and control tokens

Token	Packet	Meaning
`NAVCOM1`	D	0 = avionics bus below 24 V (radio COM and NAV displays blank), 1 = on
`BPWR`	D	Backlight power for the Servo, Stepper and Dual Stepper: 1 = main bus 20 V or more, 0 = backlight off. Sent first in the Servo and Stepper packets and after `PWR` in the Dual Stepper packet
`PWR`	D	Dual stepper board: 1 = all four OLEDs on, 0 = blank them (main bus below 24 V). Sent first in the packet
`ZULU`	D	Zulu time as HHMM, for example 1430 (dual stepper clock OLED)
`TEMP`	D	Temperature text for the dual stepper's temperature OLED, for example 25C
`B`	C	Backlight brightness 0 to 15 (0 off, 15 full). Servo, Stepper and Dual Stepper only; the Radio parses it but does nothing and the Upper ignores it
`SWING<CODE>`	C	Runs one gauge's start-up sweep on demand, for example `C,SWINGTQ:1` (stepper boards) or `C,SWINGPITCH:1` (servo board); the value is ignored
`OLEDTEST`	C	1 fills the four OLEDs solid white, 0 restores them (dual stepper board)
`KEEPALIVE`	C	Ignored command that still counts as received data and restarts the 30 second no-data timer (SwingTester sends it during a run)
`RESET`	C	Reboots the board about 2 seconds later (servo, stepper, dual stepper, radio and upper controllers on port 13136; USB HID board on port 4210). The radio and upper controllers use a software reset because a watchdog reset hung the radio's bootloader; the upper controller's reset has not been bench-tested
`MATRIX`	C	Radio controller: print the raw button matrix to the debug listener